

2009

FORMATION SHELL SCRIPTS



**GROUPE
LOGWARE**

Certifié ISO 9001 : 2000

58A rue du dessous des berges 75013 Paris

Tél. : 00 33 1 53 94 71 20 / Fax : 00 33 1 53 94 71 29

N° Organisme formation : 11754353975

Email : formation@logware.fr

Site : www.logware.fr





PRESENTATION DE LA FORMATION

OBJECTIFS

- S'initier la programmation Shell
- Acquérir une véritable autonomie dans l'écriture de scripts en Shell dans des domaines d'applications concrets (surveillance, automatisation, installation logicielle, traitement des fichiers, ...)

PUBLICS

- Informaticiens chargés de développer des Shell-scripts
- Correspondants informatiques ayant la responsabilité de grosses applications
- Responsables de l'administration ou de l'exploitation

PRE-REQUIS

- Connaissances de base d'un système Linux/Unix et de la programmation
- Expérience souhaitable de l'utilisation d'un de ce système





PROGRAMME

1. Présentation orale du formateur

2. Objectifs de la formation

- S'initier à la programmation Shell
- Acquérir une véritable autonomie dans l'écriture de scripts en Shell dans des domaines d'applications concrets (surveillance, automatisation, installation logicielle, traitement des fichiers, ...).

3. Plan :

- Présentation et rappels
- Programmation par scripts
- Mécanismes de base
- Fonctionnement en interactif
- Construction de Shell-scripts portables (ksh/bash)
- Appel d'un Shell-script
- Préambule du Shell-script
- Post ambule et retour de Shell-script
- Structures de contrôle du Shell
- Commandes internes et externes
- Mécanismes complémentaires /Debugging d'un Shell-script
- Robustesse d'un Shell-script
- Autres points



- Extensions du Korn Shell et Bash
- Outils supplémentaires / Outils d'assistance pour la création de scripts
- Manipulation de flux de texte avec sed
- Automatisation de tâches avec awk

4. Organisation pratique / horaires

- Durée : 3 jours
- Pause : 15 mn à mi-parcours le matin et l'après-midi
- Pause-déjeuner : 1 heure vers 12h30
- Fin de la formation : vers 17h

5. Tour de table

- Nom, Prénom
- Parcours
- Domaine d'activité
- Rôle
- Expérience dans le domaine
- Attentes
- Questions



SOMMAIRE

PAGES

1. PRESENTATION ET RAPPELS.....	
1.1. Principes.....	
1.2. Les différents interpréteurs : Bourne Shell, Korn Shell, Bash, C Shell, Tcsh.....	
1.3. Disponibilité des interpréteurs sur les divers systèmes.....	
1.4. Le point sur la normalisation (impacts sur l'écriture des scripts).....	
1.5. Les apports GNU (gawk, gsed...).	
1.6. Différences Bourne Shell/Korn Shell/Bash.....	
2. PROGRAMMATION PAR SCRIPTS.....	
2.1. Outils de développement.....	
2.2. Mécanisme d'exécution des scripts.....	
2.3. Règles de recherche des commandes.....	
2.4. Principes d'exécution d'une commande (exec, pipeline, sous-shell, background, ...).	
3. MECANISMES DE BASE.....	
3.1. Lecture et analyse de la ligne de commande.....	
3.2. Expansion des accolades, développement du tilde, remplacement des paramètres.....	
3.3. Substitution des commandes et évaluation arithmétique.....	
3.4. Procédés d'échappement (banalisation).....	
3.5. Les redirections (entrée et sortie standard, fichiers, tubes, document en ligne).....	
4. FONCTIONNEMENT EN INTERACTIF.....	
4.1. Invocation du shell (options).....	
4.2. Les différents fichiers de démarrage.....	



4.3. Notions d'environnement (variables, alias, fonctions).	
4.4. Historique et rappel des commandes.	
4.5. Contrôle de jobs.	
4.6. La complémentation des noms.	
4.7. Terminaison du shell.	
5. CONSTRUCTION DE SHELL-SCRIPTS PORTABLES (KSH/BASH)	
5.1. Interface avec un shell-script.	
5.2. Structure d'un shell-script.	
5.3. Options et arguments	
6. PREAMBULE DU SHELL-SCRIPT.....	
6.1. Qui interprète le shell-script ?	
6.2. Commentaires.....	
6.3. Paramètres de position (initialisation, sauvegarde, décalages).....	
6.4. Variables locales.	
6.5. Variables globales.....	
6.6. Déclaration et visibilité des fonctions.	
7. POSTAMBULE ET RETOUR DE SHELL-SCRIPT.....	
7.1. Sortie du shell-script.	
7.2. Fonction de sortie.	
7.3. Conventions utilisées.	
7.4. Valeur de retour.....	
7.5. Enchaînement de shell-script.....	
8. STRUCTURES DE CONTROLE DU SHELL	
8.1. Commandes simples, pipelines, et listes de pipelines.....	
8.2. Commandes composées, sous-shells et fonctions.	
8.3. Mécanismes de sélection et d'itération.	
8.4. Menus.	



9. COMMANDES INTERNES ET EXTERNES.....	
9.1. Entrées/Sorties.	
9.2. Interactions avec le système.....	
9.3. Arguments en ligne de commande.....	
9.4. Opérations de tests.....	
10. MECANISMES COMPLEMENTAIRES / DEBUGGING D'UN SHELL-SCRIPT	
10.1. Commandes de debugging.	
10.2. Signaux de trace et journalisation.	
11. ROBUSTESSE D'UN SHELL-SCRIPT.....	
11.1. Vérifier l'initialisation des variables.	
11.2. Gestion avancée des arguments en ligne de commande (getopts).	
12. AUTRES POINTS	
12.1. Cas particulier d'exécution d'un shell-script par cron.	
12.2. La commande eval.	
13. EXTENSIONS DU KORN SHELL ET BASH	
13.1. Tableaux de variables. Notations spécifiques.	
13.2. Opérations arithmétiques. Les alias suivis.....	
14. OUTILS SUPPLEMENTAIRES / Outils d'assistance pour la création de scripts	
14.1. Expressions rationnelles : outil grep.....	
14.2. Recherche et traitement de fichiers : outil find.....	



15. MANIPULATION DE FLUX DE TEXTE AVEC SED	
16. AUTOMATISATION DE TACHES AVEC AWK	
16.1. Eléments généraux de programmation avec awk.	
16.2. Utilisation des variables et des fonctions.	
16.3. Présentation des fonctions intégrées : mathématique, traitement de chaîne, interaction avec le système... ..	
16.4. Exemples complets de scripts Awk (statistiques système, calculs...).	



1. PRESENTATION ET RAPPELS

1.1. PRINCIPES

Sous Unix, on appelle *Shell* l'interpréteur de commandes qui fait office d'interface entre l'utilisateur et le système d'exploitation. Les Shells sont des interpréteurs : cela signifie que chaque commande saisie par l'utilisateur (ou lue à partir d'un fichier) est syntaxiquement vérifiée puis exécutée.

1.2. LES DIFFÉRENTS INTERPRÉTEURS : BOURNE SHELL, KORN SHELL, BASH, C SHELL, TCSH...

Il existe de nombreux Shells qui se classent en deux grandes familles :

- la famille *C Shell* (ex : **csh**, **tcsh**)
- la famille *Bourne Shell* (ex : **sh**, **bash**, **ksh**).

zsh est un Shell qui contient les caractéristiques des deux familles précédentes. Néanmoins, le choix d'utiliser un Shell plutôt qu'un autre est essentiellement une affaire de préférence personnelle ou de circonstance. En connaître un, permet d'accéder aisément aux autres.

Lorsque l'on utilise le système *GNU/Linux* (un des nombreux systèmes de la galaxie Unix), le Shell par défaut est **bash** (*Bourne Again Shell*). Ce dernier a été conçu en 1988 par Brian Fox dans le cadre du projet GNU2. Aujourd'hui, les développements de **bash** sont menés par Chet Ramey.

1.3. DISPONIBILITE DES INTERPRETEURS SUR LES DIVERS SYSTEMES

Les systèmes modernes disposent de plusieurs interpréteurs

1.4. LE POINT SUR LA NORMALISATION (IMPACTS SUR L'ECRITURE DES SCRIPTS)

Le besoin de portabilité des scripts a conduit à une normalisation des scripts sous le Standard de Shell IEEE 1003.2 POSIX.

1.5. LES APPORTS GNU (GAWK, GSED...)

Ces utilitaires ont été créés dans le but de remplacer les commandes **grep** et **sed**. Sa grande souplesse lui a permis de connaître un succès immédiat. Et de nouvelles versions sont apparues au fil du temps : **nawk** et **gawk** aujourd'hui.



Aujourd'hui encore, Gawk est toujours utilisé du fait de sa ressemblance avec le langage C, de sa souplesse et de sa présence sur la majorité des systèmes d'exploitation Unix. Il est encore utilisé en administration système et dans les scripts Shell en tant que commande.

1.6. BOURNE SHELL VS KORN SHELL/BASH

Voici un tableau récapitulatif :

	Bourne Shell	C-shell	K-shell	Bash
Type caractère	X	X	X	X
Type entier		Opérateur @	X	X
Type tableau		X	X	X
Variables locales	X		X	X
Fonctions	X		X	X
Traitement des chaînes	expr, cut	expr, cut	X	X
Arithmétique entière	Expr	X	X	X
Alias de commandes	X	X	X	X
Traçage à l'exécution	X	X	X	X
Contrôle des jobs		X	X	X
Historique des commandes		X	X	X
Complétion de l'historique				X

On constate que le bash est relativement complet.

2. PROGRAMMATION PAR SCRIPTS

Les Shells ne sont pas seulement des interpréteurs de commandes mais également de véritables langages de programmation.

Un Shell comme le **bash** intègre les notions de :

- ✓ *variable,*
- ✓ *d'opérateur arithmétique,*
- ✓ *de structure de contrôle,*
- ✓ *de fonctions, etc,*

présentes dans tout langage de programmation classique, mais aussi des opérateurs spécifiques (ex : |, ;).



Ex : `$ a=5` => affectation de la valeur 5 à la variable *a*

`$`

`$ echo $((a +3 ))` => affiche la valeur de l'expression *a+3*

`8`

`$`

L'opérateur (`|`), appelé *pipe*, est un opérateur caractéristique des Shells et connecte la sortie d'une commande à l'entrée de la commande suivante.

2.1. OUTILS DE DEVELOPPEMENT

Il existe sur le marché des IDE de développement Shell mais n'importe quel éditeur de texte fait l'affaire.

- ✓ Emacs
- ✓ Gedit
- ✓ KDevelop
- ✓ Kwrite
- ✓ Vi
- ✓ Etc.

2.2. MECANISME D'EXECUTION DES SCRIPTS

Un script Shell est défini par

- ✓ Fichier ASCII avec des droits d'exécutions
- ✓ Une série de commandes SHELL

Entête : `#!/bin/bash` → force l'exécution du script bash

2.3. REGLE DE RECHERCHE DES COMMANDES (MAN)

Le Man est un système d'aide en ligne permettant d'avoir d'est information sur une commande chaque nouvelle commande doit fournir un manuel de référence.

Syntaxe : `man nom_cmd`



Ex :

La page du manuel pour la commande banner donne :

banner(1) banner(1)

NAME

banner - make posters in large letters

SYNOPSIS

banner strings

DESCRIPTION

banner prints its arguments (each up to 10 characters long) in large letters on the standard output.

Each argument is printed on a separate line. Note that multiple-word arguments must be enclosed in quotes in order to be printed on the same line.

ExempleS

Print the message ``Good luck Susan'' in large letters on the screen:

banner "Good luck" Susan

2.4. PRINCIPES D'EXECUTION D'UNE COMMANDE (EXEC, PIPELINE, SOUS-SHELL, BACKGROUND, ...).

La syntaxe générale d'une commande (unix ou de **bash**) est la suivante :

[chemin/]nom_cmd [option ...] [argument ...]



C'est une suite de *mots* séparés par un ou plusieurs *blancs*. On appelle *blanc* un caractère **tab** (*tabulation horizontale*) ou un caractère **espace**.

Un *mot* est une suite de caractères non blancs. Cependant, plusieurs caractères ont une signification spéciale pour le Shell et provoquent la fin d'un mot : ils sont appelés *méta-caractères* (ex : |, <).

Bash utilise également des *opérateurs de contrôle* (ex : (, |)) et des *mots réservés* pour son propre fonctionnement :

```
! case
      do
      done
esac

      if
      else

      fi fo
      function if
      in select then
time until while { } [[ ]]
```

Bash distingue les **caractères majuscules des caractères minuscules**.

Le *nom* de la commande est le plus souvent le premier mot.

Une *option* est généralement introduite par le caractère **tiret** (ex : **-a**) ou dans la syntaxe GNU par deux caractères **tiret** consécutifs (ex : **--version**). Elle précise un fonctionnement particulier de la commande.

La syntaxe **[elt ...]** signifie que l'élément *elt* est facultatif (introduit par la syntaxe **[elt]**) ou bien présent une ou plusieurs fois (syntaxe *elt ...*). Cette syntaxe ne fait pas partie du Shell ; elle est uniquement descriptive (méta-syntaxe).

Les *arguments* désignent les objets sur lesquels doit s'exécuter la commande.

Ex : ls -l RepC RepD : commande **ls** avec l'option **l** et les arguments *RepC* et *RepD*

Lorsque l'on souhaite connaître la syntaxe ou les fonctionnalités d'une commande *cmd* (ex : **ls** ou **bash**) il suffit d'exécuter la commande **man cmd** (ex : **man bash**). L'aide en ligne de la commande *cmd* devient alors disponible.



2.4.1. MODES D'EXECUTION D'UNE COMMANDE

Deux modes d'exécution peuvent être distingués :

- ✓ l'exécution séquentielle
- ✓ l'exécution en arrière-plan.

Exécution séquentielle

Le mode d'exécution par défaut d'une commande est l'exécution séquentielle : le Shell lit la commande entrée par l'utilisateur, l'analyse, la prétraite et si elle est syntaxiquement correcte, l'exécute.

Une fois l'exécution terminée, le Shell effectue le même travail avec la commande suivante.

L'utilisateur doit donc attendre la fin de l'exécution de la commande précédente pour que la commande suivante puisse être exécutée : on dit que l'exécution est synchrone.

Si on tape la suite de commandes : *sleep 3 entrée date entrée* où *entrée* désigne la touche entrée, l'exécution de *date* débute après que le délai de 3 secondes se soit écoulé.

Pour lancer l'exécution séquentielle de plusieurs commandes sur la même ligne de commande, il suffit de les séparer par un caractère ;

Ex : `$ cd /tmp ; pwd ; echo bonjour ; cd ; pwd`
/tmp => affichée par l'exécution de pwd
bonjour => affichée par l'exécution de echo bonjour
/home/logware => affichée par l'exécution de pwd
`$`

Pour terminer l'exécution d'une commande lancée en mode synchrone, on appuie simultanément sur les touches CTRL et C (notées control-C ou ^C).

En fait, la combinaison de touches appropriée pour arrêter l'exécution d'une commande en mode synchrone est indiquée par la valeur du champ *intr* lorsque la commande *unix stty* est lancée :

Ex : `$ stty -a`
`speed 38400 baud; rows 24; columns 80; line = 0;`
`intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol = <undef>;`
`...`
`$`

Supposons que la commande *cat* soit lancée sans argument, son exécution semblera figée à un utilisateur qui débute dans l'apprentissage d'un système Unix.



Il pourra utiliser la combinaison de touches mentionnée précédemment pour terminer l'exécution de cette commande.

Ex : \$ cat
^C
\$

Exécution en arrière-plan

L'exécution en arrière-plan permet à un utilisateur de lancer une commande et de récupérer immédiatement la main pour lancer « en parallèle » la commande suivante (parallélisme logique). On utilise le caractère & pour lancer une commande en arrière-plan.

Dans l'exemple ci-dessous, la commande sleep 5 (suspendre l'exécution pendant 5 secondes) est lancée en arrière-plan. Le système a affecté le numéro d'identification 696 au processus correspondant tandis que bash a affecté un numéro de travail (ou numéro de job) égal à 1 et affiché [1]. L'utilisateur peut, en parallèle, exécuter d'autres commandes (dans cet exemple, il s'agit de la commande ps). Lorsque la commande en arrière-plan se termine, le Shell le signale à l'utilisateur après que ce dernier ait appuyé sur la touche entrée.

Ex : \$ sleep 5 &
[1696
] \$ ps
PID TTY TIME CMD
683 pts/0 00:00:00 bash
696 pts/0 00:00:00 sleep
697 pts/0 00:00:00 ps
\$ => l'utilisateur a appuyé sur la touche entrée mais sleep n'était pas terminée
\$ => l'utilisateur a appuyé sur la touche entrée et sleep était terminée
[1]+ Done sleep 5
\$

Ainsi, outre la gestion des processus spécifique à Unix, bash introduit un niveau supplémentaire de contrôle de processus. En effet, bash permet de stopper, reprendre, mettre en arrière-plan un processus, ce qui nécessite une identification supplémentaire (numéro de job).

L'exécution en arrière-plan est souvent utilisée lorsqu'une commande est gourmande en temps CPU (ex : longue compilation d'un programme).



3. MECANISME DE BASE

3.1. LECTURE ET ANALYSE DE LA LIGNE DE COMMANDE

Exercice 1 :

1.) A l'aide d'un éditeur de texte, créer un fichier *premier* contenant les lignes suivantes :

```
#!/bin/bash
# premier
echo -n "La date du jour est: "
date
```

La notation **#!** en première ligne d'un fichier interprété précise au Shell courant quel interpréteur doit être utilisé pour exécuter le script Shell (dans cet exemple, il s'agit de **/bin/bash**).

La deuxième ligne est un commentaire.

2.) Vérifier le contenu de ce fichier.

Ex : \$ **cat premier**

```
#!/bin/bash
# premier
echo -n "La date du jour est: "
date
$
```

3.) Pour lancer l'exécution d'un fichier Shell *fich*, on peut utiliser la commande : **bash fich**

Ex : \$ **bash premier**

```
la date du jour est : dimanche 3 septembre 2006, 15:41:26 (UTC+0200)
$
```

4.) Il est plus simple de lancer l'exécution d'un programme Shell en tapant directement son nom, comme on le ferait pour une commande unix ou une commande interne. Pour que cela soit réalisable, deux conditions doivent être remplies :

- l'utilisateur doit posséder les permissions **r** (*lecture*) et **x** (*exécution*) sur le fichier Shell
- le répertoire dans lequel se trouve le fichier Shell doit être présent dans la liste des chemins contenue dans **PATH**.



Ex : \$ **ls -l premier**

```
-rw-r--r-- 1 logware logware 63 sep 3 15:39 premier
```

\$

Seule la permission **r** est possédée par l'utilisateur.

Pour ajouter la permission **x** au propriétaire du fichier *fich*, on utilise la commande
chmod u+x fich

Ex : \$ **chmod u+x premier**

\$

\$ **ls -l premier**

```
-rwxr--r-- 1 logware logware 63 sep 3 15:39 premier
```

\$

Si on exécute *premier* en l'état, une erreur d'exécution se produit.

Ex : \$ **premier**

```
-bash: premier: command not found => Problème !
```

\$

Cette erreur se produit car **bash** ne sait pas où se trouve le fichier *premier*.

Ex : \$ **echo \$PATH** => affiche la valeur de la variable *PATH*

```
/bin:/usr/bin:/usr/bin/X11
```

\$

\$ **pwd**

```
/home/logware
```

\$

Le répertoire dans lequel se trouve le fichier *premier* (répertoire */home/logware*) n'est pas présent dans la liste de **PATH**. Pour que le Shell puisse trouver le fichier *premier*, il suffit de mentionner le chemin permettant d'accéder à celui-ci.

Ex : \$ **./premier**

```
la date du jour est : dimanche 3 septembre 2006, 15:45:28 (UTC+0200)
```

\$

Pour éviter d'avoir à saisir systématiquement ce chemin, il suffit de modifier la valeur de **PATH** en y incluant, dans notre cas, le répertoire courant (.).

Ex : \$ **PATH=\$PATH:.** => ajout du répertoire courant dans *PATH*

\$

\$ **echo \$PATH**

```
/bin:/usr/bin:/usr/bin/X11:.
```

\$

\$ **premier**

```
la date du jour est : dimanche 3 septembre 2006, 15:47:38 (UTC+0200)
```

\$



Exercice 2 : Ecrire un programme Shell *repcour* qui affiche le nom de connexion de l'utilisateur et le chemin absolu de son répertoire courant de la manière suivante :

```
Ex : $ repcour
      mon nom de connexion est : logware
      mon repertoire courant est : /home/logware
      $
```

3.2. EXPANSION DES ACCOLADES, DEVELOPPEMENT DU TILDE, REMPLACEMENT DES PARAMETRES

Dans la terminologie du Shell, un *paramètre* désigne toute entité pouvant contenir une valeur. Le Shell distingue trois classes de paramètres :

– les *variables*, identifiées par un *nom*

Ex : *a* , *PATH*

– les *paramètres de position*, identifiés par un *numéro*

Ex : *0* , *1* , *12*

– les *paramètres spéciaux*, identifiés par un *caractère spécial*

Ex : *#* , *?* , *\$*

Le mode d'affectation d'une valeur est spécifique à chaque classe de paramètres. Suivant celle-ci, l'affectation sera effectuée par l'utilisateur, par **bash** ou bien par le système. Par contre, pour obtenir la valeur d'un paramètre, on placera toujours le caractère **\$** devant sa référence, et cela quelle que soit sa classe.

```
Ex : $ echo $PATH => affiche la valeur de la variable PATH
      /usr/local/bin:/usr/bin:/bin:/usr/bin/X11:/usr/games:./home/logware/bin
      $ echo $$ => affiche la valeur du paramètre spécial $
      17286
      $
```

Variables

Une variable est identifiée par un *nom*, c'est-à-dire une suite de lettres, de chiffres ou de caractères **espace souligné** ne commençant pas par un chiffre. Les lettres majuscules et minuscules sont différenciées.



Les variables peuvent être classées en trois groupes :

- les variables utilisateur (ex : *a*, *valeur*)
- les variables prédéfinies du Shell (ex : **PS1**, **PATH**, **REPLY**, **IFS**)
- les variables prédéfinies de commande unix (ex : **TERM**).

En général, les noms des variables utilisateur sont en lettres minuscules tandis que les noms des variables prédéfinies (du Shell ou de commandes unix) sont en majuscules.

L'utilisateur peut affecter une valeur à une variable en utilisant

- l'opérateur d'affectation =
- la commande interne **read**.

Affectation directe

Syntaxe : *nom*=[*valeur*] [*nom*=[*valeur*] ...]

Il est impératif que le nom de la variable, le symbole = et la valeur à affecter ne forment qu'une seule chaîne de caractères.

Plusieurs affectations peuvent être présentes dans la même ligne de commande.

Ex : \$ **x=coucou y=bonjour** => la variable x contient la chaîne de caractères *coucou*
\$ => la variable y contient la chaîne *bonjour*

Tilde

Raccourci les noms des répertoires

- ~ HOME répertoire
- ~+ repertoire courant (\$PWD)
- ~- repertoire precedent (\$OLDPWD)

3.3. SUBSTITUTION DES COMMANDES ET EVALUATION ARITHMETIQUE

✓ Présentation

Syntaxe : \$(*cmd*)

Une commande *cmd* entourée par une paire de parenthèses () précédées d'un caractère \$ est exécutée par le Shell puis la chaîne \$(*cmd*) est remplacée par les résultats de la commande *cmd* écrits sur la sortie standard, c'est à dire l'écran. Ces résultats peuvent alors être affectés à une variable ou bien servir à initialiser des paramètres de position.

Ex : \$ **pwd**

/home/logware => résultat écrit par *pwd* sur sa sortie standard

\$ **repert=\$(pwd)** => la chaîne /home/logware remplace la chaîne \$(*pwd*)

\$

\$ **echo mon repertoire est \$repert**

mon repertoire est /home/logware

\$



Exercice 1 : En utilisant la substitution de commande, écrire un fichier Shell *mach* affichant :

"Ma machine courante est *nomdelamachine*"

Ex : `$ mach`

Ma machine courante est jade

\$

Lorsque la substitution de commande est utilisée avec la commande interne **set**, l'erreur suivante peut se produire :

`$ set $(ls -l .bashrc)`

bash: set: -w: invalid option

set: usage: set [--abefhkmnptuvxBCHP] [-o option] [arg ...]

`$ ls -l .bashrc`

-rw-r--r-- 1 logware users 1342 nov 29 2004 .bashrc

\$

Le premier mot issu de la substitution commence par un caractère **tiret** : la commande interne **set** l'interprète comme une option, ce qui provoque une erreur. Pour résoudre ce type de problème, on double le caractère **tiret**.

Ex : `$ set -- $(ls -l .bashrc)`

\$

`$ echo $1`

-rw-r--r--

\$

Exercice 2 : Ecrire un programme Shell *taille* qui prend un nom de fichier en argument et affiche sa taille. On ne considère aucun cas d'erreur.

Plusieurs commandes peuvent être présentes entre les parenthèses.

Ex : `$ pwd ; whoami`

/home/logware

logware

\$

`$ set $(pwd ; whoami)`

\$

`$ echo $2: $1`

logware: /home/logware

\$

Exercice 3 : A l'aide de la commande unix *date*, écrire un programme Shell *jour* qui affiche le jour courant du mois

Ex : `$ date`

dimanche 22 octobre 2006, 18:33:38 (UTC+0200)

\$

`$ jour`

22

\$



Exercice 4 : a) Ecrire un programme Shell *heure1* qui affiche l'heure sous la forme *heures:minutes:secondes*

Ex : \$ *heure1*
18:33:38
\$

b) Ecrire un programme Shell *heure* qui n'affiche que les heures et minutes

Ex : \$ *heure*
18:33
\$

Exercice 5 : Ecrire un programme Shell *nbconnect* qui affiche le nombre d'utilisateurs connectés sur la machine locale

Rq : plusieurs solutions sont possibles

solution 1, avec **who | wc -l**

solution 2, avec **uptime**

solution 3, avec **users**

solution 4, avec **who -q**

Les substitutions de commandes peuvent être imbriquées.

Ex : \$ *ls .gnome*
application-info gnome-vfs mime-info
\$
\$ *set \$(ls \$(pwd)/.gnome)*
\$
\$ *echo \$# => nombre d'entrées du répertoire .gnome*

Plusieurs sortes de substitutions (de commandes, de variables) peuvent être présentes dans la même commande.

Ex : \$ *read rep*
gnome
\$
\$ *set \$(ls \$(pwd)/\$rep) => substitutions de deux commandes et d'une*
\$ *=> variable*

La substitution de commande permet également de capter les résultats écrits par un fichier Shell.

Ex : \$ *jour*
22
\$
\$ *resultat=\$(jour)*
\$

La variable *resultat* contient la chaîne 22.

Exercice 6 : Ecrire un programme Shell *uid* qui affiche l'uid de l'utilisateur. On utilisera la commande unix **id**, la commande interne **set** et la variable prédéfinie **IFS**.



Rq : Il existe pour la substitution de commande une syntaxe plus ancienne ``cmd`` (deux caractères **accent grave** entourent la commande `cmd`), à utiliser lorsque se posent des problèmes de portabilité.

Valeur d'une expression arithmétique

La commande interne `(( expr_arith ))` n'affiche pas sur la sortie standard la valeur de l'expression arithmétique *expr_arith*.

Pour obtenir la valeur de l'expression arithmétique, on utilise la syntaxe : `$(( expr_arith ))`

Ex : `echo $(( 7 * 2 )) echo $(( a= 12*8 )) echo $(( 7 < 10 ))`

Substitutions de commandes et substitutions de paramètres peuvent être présentes dans une commande interne `((` ou bien dans une substitution d'expression arithmétique `$( ( )` si le résultat aboutit à un nombre entier.

Par exemple, dans l'expression $(((ls -l \mid wc -l) - 1))$ le résultat du pipeline $ls -l \mid wc -l$ peut être interprété comme un nombre.

Attention : lors de l'évaluation d'une expression arithmétique, les débordements éventuels ne sont pas détectés.

Ex : `$ echo $((897655*785409*56789*67899999999999999999))`
-848393034087410691 => nombre négatif !

Si l'on souhaite utiliser de grands entiers (ou des nombres réels), il est préférable d'utiliser la commande unix **bc**.

Ex : \$ bc -q
897655*785409*56789*678999999999999999999999
271856250888322242449959962260546638845
\$

L'option **-q** de **bc** évite l'affichage du message de bienvenue lorsqu'on utilise cette commande de manière interactive.

Basée sur le contexte, l'interprétation des expressions arithmétiques est particulièrement souple en **bash**.

```
Ex: $ declare -i x=35
$
$ z=x+5 => affectation de la chaîne x+5 à la variable z
$ echo $z
x+5 => non évaluée car z de type chaîne de caractères par défaut
$
```



```
$ echo $((z)) => par le contexte, z est évaluée comme une variable de type entier
40
$ (( z = z+1))
$ echo $z
```

3.4. PROCÉDES D'ÉCHAPPEMENT (BANALISATION)

Caractère d'échappement [anti-slash](#) ("\")

Anti-slash en fin de ligne

A la fin d'une ligne, un [anti-slash](#) indique que la commande continue à la ligne suivante. Cette fonction est particulièrement utile pour les grandes commandes afin de les rendre plus facilement lisibles.

Anti-slash pour former un des caractères spéciaux du C /

Les chaînes ayant un format analogue à '\$\n' sont interprétées d'une façon particulière par le bash. Elles sont transformées en conformité avec les règles du [C ANSI](#).

Exemple : `echo $'\a' # Provoquera un bip sonore.`

Voici la liste :

Échappement par antislash	Transformation par l'interpréteur bash
\a	Bip sonore
\b	Espacement arrière
\e	Échappement
\f	Saut de page (le nom anglais de ce caractère est <i>form feed</i>) ²
\n	Saut de ligne
\r	Retour chariot
\t	Caractère de tabulation horizontale
\v	Caractère de tabulation verticale
\\	Anti-slash
\'	Une apostrophe (le nom anglais de ce caractère est <i>quote</i>)
\nnn	Le caractère 8-bits dont la valeur en octal est nnn
\xHH	Le caractère 8-bits dont la valeur en hexadécimal est HH



\cx	Le caractère contrôle-X
-----	-------------------------

Anti-slash avant un des meta-caractères du bash [

Les meta-caractères, notamment "*" (étoile), sont interprétés par l'interpréteur bash (le plus souvent remplacement par d'éventuels fichiers), ce qui est gênant dans certains cas (commande *find*³, [sed](#), ...etc).

Exemple :

```
cd /etc;find . -name r*
```

Le message d'erreur sera

Find: Les chemins doivent précéder l'expression

Une des solutions consiste à utiliser un anti-slash avant le caractère "*".

Exemple : `cd /etc;find . -name r\*`

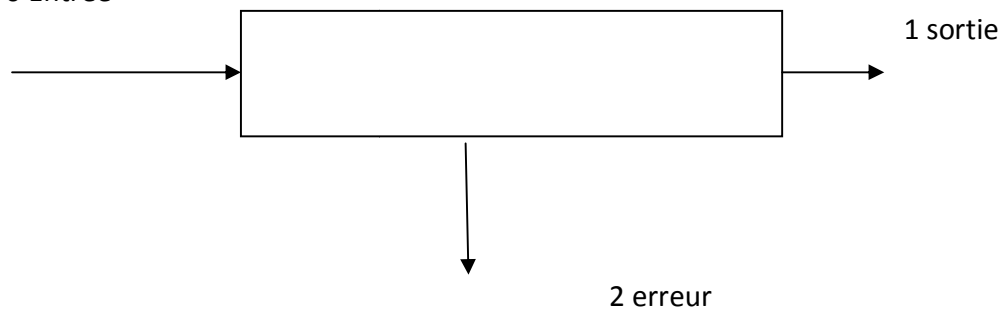
3.5. LES REDIRECTIONS (ENTREE ET SORTIE STANDARD, FICHIERS, TUBES, DOCUMENT EN LIGNE)

Descripteurs de fichiers

Un processus Unix possède par défaut trois voies d'interaction avec l'extérieur appelées *entrées / sorties standard* identifiées par un entier positif ou nul appelé *descripteur de fichier*. Ces entrées / sorties standard sont :

- une *entrée standard* , de descripteur **0**
- une *sortie standard* , de descripteur **1**
- une *sortie standard pour les messages d'erreurs*, de descripteur **2**.

0 Entrée



Toute commande étant exécutée par un processus, nous dirons également qu'une commande possède trois entrées / sorties standard.

0 2 1 commande

De manière générale, une commande de type filtre (ex : **cat**) prend ses données sur son entrée standard qui correspond par défaut au clavier, affiche ses résultats sur sa sortie standard, par défaut l'écran, et affiche les erreurs éventuelles sur sa sortie standard pour les messages d'erreurs, par défaut l'écran également.

Redirections élémentaires

On peut rediriger séparément chacune des trois entrées/sorties standard d'une commande. Cela signifie qu'une commande pourra

- lire les données à traiter à partir d'un fichier et non du clavier de l'utilisateur
- écrire les résultats ou erreurs dans un fichier et non à l'écran.

Redirection de la sortie standard : > *fichier* ou 1> *fichier*

Ex : \$ **pwd**

/home/logware

\$ **pwd** > *fich*

\$ => aucun résultat affiché à l'écran !

\$ **cat** *fich* => le résultat a été enregistré dans le fichier *fich*

/home/logware

\$

Cette première forme de redirection de la sortie standard écrase l'ancien contenu du fichier de sortie (fichier *fich*) si celui-ci existait déjà, sinon le fichier est créé.

Le Shell prétraite une commande avant de l'exécuter : dans l'exemple ci-dessous, le Shell crée un fichier vide *f* devant recevoir les résultats de la commande (ici inexistante) ; l'unique effet de >*f* sera donc la création du fichier *f* ou sa remise à zéro.

Ex : \$ >*f* => crée ou remet à zéro le fichier *f*

\$

\$ **ls -l** *f*

-rw-r--r-- 1 logware users 0 oct 22 11:34 *f*

\$

Une commande ne possède qu'une seule sortie standard; par conséquent, si on désire effacer le contenu de plusieurs fichiers, il est nécessaire d'utiliser autant de commandes que de fichiers à effacer. Le caractère ; permet d'exécuter séquentiellement plusieurs commandes écrites sur la même ligne.

Ex : \$ >*f1* ; >*f2*

\$

La redirection de la sortie standard de **cat** est souvent utilisée pour affecter un contenu succinct à un fichier.



Ex : `cat >tata` => l'entrée standard est directement recopiée dans *tata*

```
$ a demain
si vous le voulez bien
^D
$
$ cat tata
a demain
si vous le voulez bien
$
```

Ocat1tataa demainsi vous le voulez bien

Pour concaténer (c'est à dire *ajouter à la fin*) la sortie standard d'une commande au contenu d'un fichier, une nouvelle forme de redirection doit être utilisée : `>> fichier`

Ex : `$ pwd > t`

```
$
$ date >> t
$
$ cat t
/home/logware
lundi 20 novembre 2006, 15:27:41 (UTC+0100)
$
```

Comme pour la redirection précédente, l'exécution de `>> fich` crée le fichier *fich* s'il n'existait pas.

Redirection de la sortie standard pour les messages d'erreur : `2> fichier`

On ne doit laisser aucun caractère **espace** entre le chiffre **2** et le symbole `>`.

Ex : `$ pwd`

```
/home/logware
$ ls vi => l'éditeur de texte vi se trouve dans le répertoire /usr/bin
ls: vi: Aucun fichier ou répertoire de ce type
$ ls vi 2> /dev/null
$
```

Le fichier spécial **/dev/null** est appelé « poubelle » ou « puits » car toute sortie qui y est redirigée, est perdue. En général, il est utilisé lorsqu'on est davantage intéressé par le code de retour de la commande plutôt que par les résultats ou messages d'erreur qu'elle engendre.

Comme pour la sortie standard, il est possible de concaténer la sortie standard pour les messages d'erreur d'une commande au contenu d'un fichier : `2>> fichier`

Ex : `$ ls vi`

```
ls: vi: Aucun fichier ou répertoire de ce type
$
$ ls vi 2>err
$
$ ls date 2>>err
$
$ cat err
ls: vi: Aucun fichier ou répertoire de ce type
ls: date: Aucun fichier ou répertoire de ce type
```



Fermeture des entrées / sorties standard :

Fermeture de l'entrée standard : <&-

Fermeture de la sortie standard : >&-

Fermeture de la sortie standard pour les messages d'erreur : 2>&-

Ex : \$ >&- echo coucou

-bash: echo: write error: Mauvais descripteur de fichier

\$

\$

Il y a fermeture de la sortie standard puis une tentative d'écriture sur celle-ci : l'erreur est signalée par l'interpréteur de commandes.

Tubes

Le mécanisme de *tube* (symbolisé par le caractère |) permet d'enchaîner l'exécution de commandes successives en connectant la sortie standard d'une commande à l'entrée standard de la commande suivante :

Ex : ls -l /bin | more

Il y a affichage du contenu du répertoire **/bin** (commande **ls -l /bin**) écran par écran (commande **more**). En effet, les informations affichées par **ls -l** sont envoyées vers l'entrée de la commande **more** qui les affiche écran par écran. La sortie standard de la dernière commande n'étant pas redirigée, l'affichage final s'effectue à l'écran.

ls -l /binmore|1 1 022

Un résultat équivalent aurait pu être obtenu d'une manière moins élégante en exécutant la suite de commandes :

ls -l /bin >temporaire ; more < temporaire ; rm temporaire

La commande **ls -l /bin** écrit le résultat dans le fichier *temporaire* ; ensuite, on affiche écran par écran le contenu de ce fichier ; enfin, on efface ce fichier devenu inutile. Le mécanisme de tube évite à l'utilisateur de gérer ce fichier intermédiaire.

Pipelines

On appelle *pipeline*, une suite non vide de commandes connectées par des tubes :

$cmd_1 | \dots | cmd_n$

Chaque commande est exécutée par un processus distinct, la sortie standard de la commande cmd_{i-1} étant connectée à l'entrée standard de la commande cmd_i .

Ex : \$ date | tee trace1 trace2 | wc -l

1 => **date** n'écrit ses résultats que sur une seule ligne

\$

\$ cat trace1

lundi 20 novembre 2006, 15:37:49 (UTC+0100)



```
$  
$ cat trace2  
lundi 20 novembre 2006, 15:37:49 (UTC+0100)  
$
```

La commande unix **tee** écrit le contenu de son entrée standard sur sa sortie standard tout en gardant une copie dans le ou les fichiers dont on a passé le nom en argument.

Dans l'exemple ci-dessus, **tee** écrit le résultat de **date** dans les fichiers *trace1* et *trace2* ainsi que sur sa sortie standard, résultat passé à la commande **wc -l**.

4. FONCTIONNEMENT EN INTERACTIF

En premier lieu, un Shell doit fournir un environnement de travail agréable et puissant. Par exemple, **bash** permet (entre autres) :

- le rappel des commandes précédentes (gestion de l'historique) ; cela évite de taper plusieurs fois la même commande
- la modification en ligne du texte de la commande courante (ajout, retrait, remplacement de caractères) en utilisant les commandes d'édition de l'éditeur de texte **vi** ou **emacs**
- la gestion des travaux lancés en arrière-plan (appelés *jobs*) ; ceux-ci peuvent être démarrés, stoppés ou repris suivant les besoins
- l'initialisation adéquate de variables de configuration (chaîne d'appel de l'interpréteur, chemins de recherche par défaut) ou la création de raccourcis de commandes (commande **alias**).

Illustrons cet ajustement de configuration par un exemple. Le Shell permet d'exécuter une commande en mode interactif ou bien par l'intermédiaire de fichiers de commandes (*scripts*). En mode interactif, **bash** affiche à l'écran une *chaîne d'appel* (appelée également *prompt* ou *invite*), qui se termine par défaut par le caractère **#** suivi d'un caractère **espace** pour l'administrateur système (utilisateur **root**) et par le caractère **\$** suivi d'un caractère **espace** pour les autres utilisateurs. Cette chaîne d'appel peut être relativement longue.

4.1. INVOCATION DU SHELL (OPTIONS).

Il existe différente façon d'activer un Shell plutôt qu'un autre

- DEFINITION
 - FICHIER /etc/passwd par (l'admin)
 - VARIABLE SHELL



- ACTIVATION
 - AU LOGIN (chargement avec le profile)
 - SUR COMMANDE sur le Shell lui
 - » bsh
 - » csh
 - » ksh
- BOURNE
 - /bin/bsh
- CSHELL
 - /bin/csh
- KORN SHELL
 - /bin/ksh

4.2. LES DIFFERENTS FICHIERS DE DEMARRAGE.

Le programme Shell /bin/bash utilise une collection de fichiers de démarrage pour aider à la création d'un environnement de travail, chaque fichier a une utilisation spécifique et peut affecter différemment la connexion et les environnements interactifs.

Un Shell interactif de connexion est lancé après une identification positive avec /bin/login en lisant le fichier /etc/passwd. Un Shell interactif, sans login, est lancé à la ligne de commande (c'est-à-dire [prompt]\$ /bin/bash). Un Shell non-interactif est présent habituellement lorsqu'un script Shell tourne. Il est non interactif parce qu'il exécute un script et n'attend pas d'entrée utilisateur entre les commandes.

Pour plus d'informations, voir `info bash -- Noeuds: Bash Startup Files et Interactive Shells`.

Les fichiers suivants sont nécessaires pour s'assurer que l'environnement correct est lu pour chacune des façons dont le Shell peut être appelé : /etc/profile, /etc/bashrc, ~/.bash_profile et ~/.bashrc. Le fichier ~/.bash_logout n'est pas utilisé pour une invocation du Shell. Il est lu par le Shell quand un utilisateur se déconnecte du système. Les fichiers /etc/profile et ~/.bash_profile sont lus quand le Shell est invoqué comme Shell interactif de connexion. Le fichier ~/.bashrc est lu quand le Shell est invoqué comme Shell interactif sans fonction de connexion.

Voici la base d'un /etc/profile. Les commentaires sur ce fichier devraient expliquer tout ce dont vous avez besoin. Pour plus d'informations sur les séquences d'échappement que vous pouvez utiliser pour votre invite (par exemple, la variable d'environnement PS1), voir `info bash – Nœud : Imprimer une invite`.



4.3. NOTIONS D'ENVIRONNEMENT (VARIABLES, ALIAS, FONCTIONS).

Le système Linux comme tout autre système d'exploitation met à dispositions des de tous (humain & processus et programme) tout au long d'une session.

VARIABLES "SYSTEMES"

- PATH » Répertoire de recherche
- HOME → Répertoire d'accueil de l'utilisateur
- TERM → Type de console
- LANG → Langage pour les commandes
- PS1 → Prompt de premier niveau

4.4. HISTORIQUE ET RAPPEL DES COMMANDES.

On peut accéder aux commandes exécuté sur le Shell :

En mode interactif → avec les touches up /down du clavier

En mode non – interactif → il existe un fichier

\$HOME/.sh_history

– STOCKAGE DES COMMANDES

– VARIABLES

» HISTSIZE (Taille du fichier)

» HISTFILE (nom du fichier)

– COMMANDE fc

» EDITION FICHIER HISTFILE

» RE-EXECUTION DES COMMANDES

- -r (RE-EXECUTE LA DERNIERE COMMANDE)

4.5. CONTROLE DE JOBS.

XXXXXX

4.6. LA COMPLEMENTATION DES NOMS.

bash est une interface relativement efficace pour entrer des commandes. En tapant le début d'une commande suivie de la touche tabulation, on peut soit compléter la commande si elle est unique, soit voir une liste de toutes les commandes commençant par la même chaîne.

Le raccourci "CTRL-r" est également particulièrement intéressant.



Création d'un alias

Pour créer un ou plusieurs alias, on utilise la syntaxe : **alias nom=valeur ...**

Ex : \$ alias cx='chmod u+x'
\$

Le nom de l'alias peut être présent dans sa propre définition. La commande *alias rm='rm -i'* redéfinit le comportement par défaut de la commande unix **rm** en demandant à l'utilisateur de confirmer la suppression (on force l'utilisateur à utiliser l'option **-i**).

Ex : \$ alias rm='rm -i'
\$
\$ > err => création du fichier *err*
\$
\$ rm err
rm: détruire fichier régulier vide `err'? o => l'alias *rm* demande
\$ => confirmation
\$ ls err
ls: err: Aucun fichier ou répertoire de ce type => le fichier *err* a été
\$ => supprimé

Attention :

On ne doit pas définir un alias et utiliser cet alias dans la même ligne mais sur deux lignes différentes.

Ex : \$ alias aff='echo bonjour' ; aff tout le monde
-bash: aff: command not found => *aff tout le monde* n'a pu s'exécuter !
\$
\$ aff la compagnie
bonjour la compagnie => l'alias est connu
\$

Si l'on désire utiliser plusieurs alias dans la même commande, il est nécessaire de laisser un caractère **espace** ou **tabulation** comme dernier caractère de *valeur*.

Ex : \$ cat /etc/debian_version
4.0
\$
\$ alias c=cat d=/etc/debian_version
\$
\$ c d
cat: d: Aucun fichier ou répertoire de ce type

Dans l'exemple ci-dessus, l'alias *d* n'a pas été interprété car le dernier caractère de la valeur de *c* n'est ni un **espace**, ni une **tabulation**. En ajoutant un caractère **espace**, on obtient le résultat voulu.



Suppression d'un alias

Pour rendre indéfinie la valeur d'un ou plusieurs alias, on utilise la commande **unalias**.

La syntaxe est : **unalias nom ...**

Ex : \$ **unalias g rep aff scd**

```
$  
$ alias  
alias c='cat '  
alias cx='chmod u+x'  
alias d='/etc/debian_version'  
alias h='set $(date) ; echo ${5%:*}'  
alias ll='ls -l'  
alias ls='ls --color=auto'  
alias rm='rm -i'
```

4.7. TERMINAISON DU SHELL

Xxxxxxxx

5. CONSTRUCTION DE SHELL-SCRIPTS PORTABLES (KSH/BASH)

5.1. INTERFACE AVEC UN SHELL-SCRIPT

Lorsqu'un traitement nécessite l'exécution de plusieurs commandes, il est préférable de les sauvegarder dans un fichier plutôt que de les retaper au clavier chaque fois que le traitement doit être lancé. Ce type de fichier est appelé *fichier de commandes* ou *fichier Shell* ou encore *script Shell*.

Exercice 1 :

- 1.) A l'aide d'un éditeur de texte, créer un fichier *premier* contenant les lignes suivantes :

```
#!/bin/bash  
# premier  
echo -n "La date du jour est: "  
date
```

La notation **#!** en première ligne d'un fichier interprété précise au Shell courant quel interpréteur doit être utilisé pour exécuter le script Shell (dans cet exemple, il s'agit de **/bin/bash**).

La deuxième ligne est un commentaire.



2.) Vérifier le contenu de ce fichier.

```
Ex : $ cat premier
#!/bin/bash
# premier
echo -n "La date du jour est: "
date
$
```

3.) Pour lancer l'exécution d'un fichier Shell *fich*, on peut utiliser la commande : **bash fich**

```
Ex : $ bash premier
la date du jour est : dimanche 3 septembre 2006, 15:41:26 (UTC+0200)
$
```

4.) Il est plus simple de lancer l'exécution d'un programme Shell en tapant directement son nom, comme on le ferait pour une commande unix ou une commande interne. Pour que cela soit réalisable, deux conditions doivent être remplies :

- l'utilisateur doit posséder les permissions **r** (*lecture*) et **x** (*exécution*) sur le fichier Shell
- le répertoire dans lequel se trouve le fichier Shell doit être présent dans la liste des chemins contenue dans **PATH**.

```
Ex : $ ls -l premier
-rw-r--r-- 1 logware logware 63 sep 3 15:39 premier
$
```

Seule la permission **r** est possédée par l'utilisateur.

Pour ajouter la permission **x** au propriétaire du fichier *fich*, on utilise la commande :

chmod u+x fich

```
Ex : $ chmod u+x premier
$
$ ls -l premier
-rwxr--r-- 1 logware logware 63 sep 3 15:39 premier
$
```

5.2. STRUCTURE D'UN SHELL-SCRIPT

Si on exécute *premier* en l'état, une erreur d'exécution se produit.

```
Ex : $ premier
-bash: premier: command not found => Problème !
$
```

Cette erreur se produit car **bash** ne sait pas où se trouve le fichier *premier*.

```
Ex : $ echo $PATH => affiche la valeur de la variable PATH
/bin:/usr/bin:/usr/bin/X11
```



```
$
$ pwd
/home/logware
$
```

Le répertoire dans lequel se trouve le fichier *premier* (répertoire */home/logware*) n'est pas présent dans la liste de **PATH**. Pour que le Shell puisse trouver le fichier *premier*, il suffit de mentionner le chemin permettant d'accéder à celui-ci.

Ex : \$./premier

```
la date du jour est : dimanche 3 septembre 2006, 15:45:28 (UTC+0200)
$
```

Pour éviter d'avoir à saisir systématiquement ce chemin, il suffit de modifier la valeur de **PATH** en y incluant, dans notre cas, le répertoire courant (.).

Ex : \$ PATH=\$PATH:. => ajout du répertoire courant dans *PATH*

```
$
$ echo $PATH
/bin:/usr/bin:/usr/bin/X11:.
$
$ premier
la date du jour est : dimanche 3 septembre 2006, 15:47:38 (UTC+0200)
$
```

Exercice 1 : Ecrire un programme Shell *repcour* qui affiche le nom de connexion de l'utilisateur et le chemin absolu de son répertoire courant de la manière suivante :

Ex : \$ repcour

```
mon nom de connexion est : logware
mon repertoire courant est : /home/logware
$
```

5.3. OPTIONS ET ARGUMENTS

Comme dans n'importe quel langage de programmation il est souhaitable de pouvoir faire passer des arguments à un script.

\$0	nom de la commande
\$ n	nieme paramètre
\$#	nombre de paramètres
\$* \$@	liste de tous les paramètres La signification de \$* et de \$@ est identique. Cependant, lorsque "\$*" est utilisé comme argument de commande, il est équivalent à "\$1 \$2 " alors que "\$@" est équivalent à "\$1" "\$2"



Pour décaler les paramètres, on peut utiliser la commande *shift*

Quelques variables spéciales

\$\$	le numéro de processus de la dernière commande
\$?	Status de la dernière commande

Exemple :

Code: file.sh

```
#!/bin/sh
```

```
echo $1
```

```
echo $2
```

```
echo $0
```

```
exit(0) ;
```

donne

arg1

arg2

file.sh

6. PREAMBULE DU SHELL-SCRIPT

Sha bang

```
# !/bin/nom_shell(bash , ksh , sh , tcsh etc...)
```

6.1. QUI INTERPRETE LE SHELL-SCRIPT ?

Le nom_sell represente l'interpreteur choisi.

Rappel (voir 4.1)

Il existe différente façon d'activer un Shell plutôt qu'un autre

- DEFINITION

- FICHER /etc/passwd par (l'admin)
- VARIABLE SHELL



- ACTIVATION
 - AU LOGIN (chargement avec le profile)
 - SUR COMMANDE sur le Shell lui
 - » bsh
 - » csh
 - » ksh
- BOURNE
 - /bin/bsh
- CSHELL
 - /bin/csh
- KORN SHELL
 - /bin/ksh

6.2. COMMENTAIRES

Un commentaire débute avec le caractère **#** et se termine avec la fin de la ligne. Un commentaire est ignoré par le Shell.

Ex : `$ echo bonjour # l'ami`
 bonjour
`$ echo coucou # l' ami`
 cou
`coucou $ # echo coucou`
`$`



Pour que le caractère # soit reconnu en tant que début de commentaire, il ne doit pas être inséré à l'intérieur d'un mot ou terminer un mot.

```
Ex : $ echo il est#10 heures
      il est#10 heures
      $
      $ echo bon# jour bon# jour
```

6.3. PARAMETRES DE POSITION (INITIALISATION, SAUVEGARDE, DECALAGES).

Paramètres de position

Un **paramètre de position** est référencé par un ou plusieurs chiffres : 8 , 0 , 23

L'assignation de valeurs à des paramètres de position s'effectue :

- soit à l'aide de la commande interne **set**
- soit lors de l'appel d'un fichier Shell ou d'une fonction Shell.

ATTENTION : on ne peut utiliser ni le symbole =, ni la commande interne **read** pour affecter directement une valeur à un paramètre de position.

```
Ex : $ 23=bonjour
      -bash: 23=bonjour: command not found
      $
      $ read 4
      aa
      -bash: read: `4': not a valid identifier
      $
```

Commande interne set

La commande interne **set** affecte une valeur à un ou plusieurs paramètres de position en numérotant ses arguments suivant leur position. La numérotation commence à **1**.

Syntaxe : **set** *arg1* ...

```
Ex : $ set alpha beta gamma => alpha est la valeur du paramètre de position 1, beta la
      $ => valeur du deuxième paramètre de position et gamma
      $ => la valeur du paramètre de position 3
```

Pour obtenir la valeur d'un paramètre de position, il suffit de placer le caractère **\$** devant son numéro ; ainsi, **\$1** permet d'obtenir la valeur du premier paramètre de position, **\$2** la valeur du deuxième et ainsi de suite.

```
Ex : $ set ab be ga => numérotation des mots ab, be et ga
      $
      $ echo $3 $2
      ga be
      $
      $ echo $4
      $
```



Tous les paramètres de position sont réinitialisés dès que la commande interne **set** est utilisée avec au moins un argument.

Ex : `$ set coucou`
 `$ echo $1`
 coucou
 `$ echo $2`
 => les valeurs *be* et *ga* précédentes ont disparues
 `$`

La commande **set --** rend indéfinie la valeur des paramètres de position préalablement initialisés.

Ex : `$ set alpha beta`
 `$ echo $1`
 alpha
 `$`
 `$ set --`
 `$ echo $1`
 => les valeurs *alpha* et *beta* sont perdues

6.4. VARIABLES LOCALES.

Une variable déclarée *localement* n'est visible qu'à l'intérieur du [bloc de code](#) dans laquelle elle apparaît. Elle a une << visibilité >> locale. Dans une fonction, une *variable locale* n'a une signification qu'à l'intérieur du bloc de la fonction.

Les variables en Shell sont identifiées par leur nom , affectés par l'opérateur « = »

» variable=valeur

VAR= « value »

» echo \$VAR

value

– Affectation d'une chaîne vide

» VAR=

» echo \$VAR

– \$VARIABLE → Valeur de la variable

– \${VAR}TABLE → Valeur de VAR suivi de "TABLE"

– \${VAR - <valeur>} → Valeur de VAR si définie sinon <valeur>

» VALEUR DE VAR SI DEFINIE

• \${VAR ? <message>} → Le contenu de var si défini sinon le contenu de message sinon sort du Shell.

– message = 'mess', ou \$message

Pour désaffecter une variable il faut faire

DESAFFECTATION

– unset VAR



6.5. VARIABLES GLOBALES.

Une variable déclarée globalement est une variable qui est visible de partout le système. C'est les cas de \$PATH, \$JAVA_HOME etc.

Le mot clé export devant une variable le rend globale.

- AFFECTATION EXTERIEURE → Pour rendre la variable globale
- export VAR=\$

6.5.1. TYPES DE DONNEES

❖ Type entier

Pour définir et initialiser une ou plusieurs variables de type entier, on utilise la syntaxe suivante :

declare -i nom[=expr_arith] [nom[=expr_arith] ...]

Ex : \$ **declare -i x=35** => définition et initialisation de la variable entière x

```
$  
$ declare -i v w => définition des variables entières v et w  
$  
$ v=12 => initialisation de v par affectation  
$  
$ read w  
34 => initialisation de w par lecture  
$
```

Rappel : Il n'est pas nécessaire de définir une variable avant de l'utiliser !

Pour que la valeur d'une variable entière ne soit pas accidentellement modifiée après qu'elle ait été initialisée, il suffit d'ajouter l'attribut r.

Ex : \$ **declare -ir a=-6**

```
$  
$ a=7  
-bash: a: readonly variable => seule la consultation est autorisée !  
$
```

Enfin, pour connaître toutes les variables entières définies, il suffit d'utiliser la commande **declare -i**.

Ex : \$ **declare -i**

```
declare -ir EUID="1007"  
declare -ir PPID="19620"  
declare -ir UID="1007"  
declare -ir a="-6"  
declare -i v="12"  
declare -i w="14"  
declare -i x="35"
```



❖ Commande interne

Cette commande interne est utilisée pour effectuer des opérations arithmétiques.

Syntaxe : `(( expr_arith ))`

Son fonctionnement est le suivant : *expr_arith* est évaluée ; si cette évaluation donne une valeur différente de **0**, alors le code de retour de la commande interne `((` est égal à 0 sinon le code de retour est égal à 1.

Il est donc important de distinguer deux aspects de la commande interne `(( expr_arith ))` :

- la valeur de *expr_arith* issue de son évaluation et
- le code de retour de `(( expr_arith ))`.

Attention : la valeur de *expr_arith* n'est pas affichée sur la sortie standard.

Ex : `$ (( -5 )) =>` la valeur de l'expression arithmétique est égale à -5 (c.-à-d.

`$ =>` différente de 0), donc le code de retour de `((` est égal à **0**

`$ echo $?`

0

`$`

`$ (( 0 )) =>` la valeur de l'expression arithmétique est 0, donc le code de retour

`$ =>` de `((` est égal à **1**

`$ echo $?`

1

`$`

Les opérateurs permettant de construire des expressions arithmétiques évaluables par **bash** sont issus du langage C (ex : `= + - > <=`).

Ex : `$ declare -i a=2 b`

`$ (( b = a + 7 ))`

`$`

Le format d'écriture est libre à l'intérieur de la commande `((`. En particulier, plusieurs caractères **espace** ou **tabulation** peuvent séparer les deux membres d'une affectation.

Il est inutile d'utiliser le caractère de substitution `$` devant le nom d'une variable car il n'y a pas d'ambiguïté dans l'interprétation ; par contre, lorsqu'une expression contient des paramètres de position, le caractère `$` doit être utilisé.

Ex : `$ date`

jeudi 21 décembre 2006, 19:41:42 (UTC+0100)

`$`

`$ set $(date)`

`$`

`$ (( b = $2 + 1 )) =>` incrémentation du jour courant

`$`

`$ echo $b`

22

`$`



Code de retour de ((et structures de contrôle

Le code de retour d'une commande interne ((est souvent exploité dans une structure de contrôle **if** ou **while**.

Le programme Shell **dix** ci-dessous affiche les dix chiffres :

```
-----  
#!/bin/bash  
    declare -i i=0  
while (( i < 10 ))  
do  
    echo $i  
    (( i = i + 1 )) # ou (( i++ ))  
done  
-----
```

Son fonctionnement est le suivant : l'expression $i < 10$ est évaluée, sa valeur est vraie, le code de retour de $((i < 10))$ est égal à 0, le corps de l'itération est exécuté. Après affichage de la valeur 9, la valeur devient égale à 10. L'expression $i < 10$ est évaluée, sa valeur est maintenant fausse, le code de retour de $((i < 10))$ est égal à 1, l'itération se termine.

❖ Protection de caractères

Le Shell utilise différents caractères particuliers pour effectuer ses propres traitements (\$ pour la substitution, > pour la redirection de la sortie standard, * comme caractère générique, etc.). Pour utiliser ces caractères particuliers comme de simples caractères, il est nécessaire de les protéger de l'interprétation du Shell. Trois mécanismes sont utilisables :

Protection d'un caractère à l'aide du caractère \

Ce caractère protège le caractère qui suit immédiatement le caractère \.

Ex : \$ echo * => le caractère * perd sa signification de caractère générique

```
*  
$ echo *  
tata toto  
$  
$ echo \\ => le deuxième caractère \ perd sa signification de caractère de protection  
\  
$ echo N\'oublie pas !  
N'oublie pas !  
$
```

Rq : dans les exemples de cette section, les caractères **en vert** ont perdu leur signification particulière, les caractères **en rouge** sont interprétés.

Le caractère \ permet également d'ôter la signification de la touche **Entrée**. Cela a pour effet d'aller à la ligne sans qu'il y ait exécution de la commande. En effet, après saisie d'une commande, l'utilisateur demande au Shell l'exécution de celle-ci en appuyant sur cette touche.



Annuler l'interprétation de la touche **Entrée** autorise l'utilisateur à écrire une longue commande sur plusieurs lignes.

Dans l'exemple ci-dessous, le Shell détecte que la commande interne **echo** n'est pas terminée ; par conséquent, **bash** affiche une chaîne d'appel différente matérialisée par un caractère **>** suivi d'un caractère **espace** invitant l'utilisateur à continuer la saisie de sa commande.

Ex : `$ echo coucou \Entrée`

```
> salut Entrée => terminaison de la commande : le Shell l'exécute !
coucou salut
$
```

Protection de caractères à l'aide d'une paire de guillemets "chaîne"

Tous les caractères de *chaîne* sauf `$ \ ` "` sont protégés de l'interprétation du Shell. Cela signifie, par exemple, qu'à l'intérieur d'une paire de guillemets le caractère `$` sera quand même interprété comme une substitution, etc.

Ex : `$ echo "< * $PWD * >"`

```
< * /home/logware * >
$
$ echo "< * \"$PWD\" * >"
< * "/home/logware" * >
$
```

Protection totale 'chaîne'

Entre une paire d'**apostrophes** (`'`), aucun caractère de *chaîne* (sauf le caractère `'`) n'est interprété.

Ex : `$ echo '< * $PWD * >'`

```
< * $PWD * >
$
```

❖ Longueur d'une chaîne de caractères

Syntaxe : `${#paramètre}`

Cette syntaxe est remplacée par la longueur de la chaîne de caractères contenue dans **paramètre**. Ce dernier peut être une variable, un paramètre spécial ou un paramètre de position.

```
Ex : $ echo $PWD
/home
/home/logware $ echo ${#PWD}
14 => longueur de la chaîne /home/logware
$
$ set "au revoir"
$ echo ${#1}
9 => la valeur de $1 étant au revoir, sa longueur est 9
```



```
$
$ ls >/dev/null
$
$ echo ${#?}
1 => contenue dans $?, la valeur du code de retour de ls
$ => est 0, par conséquent la longueur est 1
```

❖ Modificateurs de chaînes

Les modificateurs de chaînes permettent la suppression d'une sous-chaîne de caractères correspondant à un modèle exprimé à l'aide de caractères ou d'expressions génériques.

Suppression de la plus courte sous-chaîne à gauche

Syntaxe : **`${paramètre#modèle}`**

```
Ex : $ echo $PWD
/home/logware
$
$ echo ${PWD#*/}
home/logware => le premier caractère / a été supprimé
$
$ set "12a34a"
$
$ echo ${1#a}
34a => suppression de la sous-chaîne 12a
$
```

Suppression de la plus longue sous-chaîne à gauche

Syntaxe : **`${paramètre##modèle}`**

```
Ex : $ echo $PWD
/home/logware
$
$ echo ${PWD##*/}
logware => suppression de la plus longue sous-chaîne à gauche se
$ => terminant par le caractère /
$ set 12a34ab
$
$ echo ${1##*a}
b
$
```

Suppression de la plus courte sous-chaîne à droite

Syntaxe : **`${paramètre%modèle}`**

```
Ex: $ echo $PWD
/home/logware
$ echo ${PWD%/*}
/home
$
```



Suppression de la plus longue sous-chaîne à droite

Syntaxe : **`${paramètre%%modèle}`**

Ex : **`$ eleve="Pierre Dupont::12:10::15:9"`**

```
$  
$ echo ${eleve%%:*}  
Pierre Dupont  
$
```

La variable *eleve* contient les prénom et nom et diverses notes d'un élève. Les différents champs sont séparés par un caractère **deux-points**. Il peut manquer des notes à un élève (cela se caractérise par un champ vide).

Exercices :

1. En utilisant les modificateurs de chaînes,

- écrire un programme Shell *touslesutil* qui lit le contenu du fichier **`/etc/passwd`** et affiche le nom des utilisateurs enregistrés.
- modifier ce programme (soit *touslesutilid*) pour qu'il affiche le nom et l'uid de chaque utilisateur.

2. En utilisant les modificateurs de chaînes,

- écrire un programme Shell *basenom* ayant un fonctionnement similaire à la commande unix **`basename`**. Cette commande affiche le dernier élément d'un chemin passé en argument. Il n'est pas nécessaire que ce chemin existe réellement.

```
Ex : $ basename /toto/tata/tutu  
tutu  
$
```

- si un suffixe est mentionné comme deuxième argument, celui ci est également ôté de l'élément par la commande **`basename`**.

```
Ex : $ basename /toto/tata/tutu/prog.c.c  
prog  
$
```

Ecrire un programme *basenom1* qui se comporte de la même manière.

3. Ecrire une commande *dirnom*, similaire à la commande unix **`dirname`** ; la commande unix **`dirname`** peut être vue comme la commande complémentaire à **`basename`** car **`dirname`** supprime le dernier élément d'un chemin; si cela donne la chaîne vide, alors le caractère **`.`** est affiché.

```
Ex : $ dirname /home/logware/bin  
/home/logware  
$  
$ dirname toto  
$
```



❖ Extraction de sous-chaînes

`${paramètre:ind}` : extrait de la valeur de ***paramètre*** la sous - chaîne débutant à l'indice ***ind***. La valeur de ***paramètre*** n'est pas modifiée.

Attention : l'indice du premier caractère d'une chaîne est **0**

Ex : `ch="abcdefghijk" echo ${ch:3}` affiche *defghijk*
01234567..10

`${paramètre:ind:nb}` : extrait ***nb*** caractères à partir de l'indice ***ind***

Ex : `$ echo ${ch:8:2}`

```
ij
$ set ABCDEFGH
$
$ echo ${1:4:3}
EFG
$
```

Exercice : a) Ecrire un programme *calibre* prenant deux arguments, une chaîne de caractères *ch* et un nombre *nb*, qui affiche les *nb* premiers caractères de *ch*. Aucune erreur ne doit être traitée.

b) Modifier le programme précédent, soit *calibre1*, pour qu'il vérifie que :

- le nombre d'arguments est correct
- le deuxième argument est un nombre (suite non vide de chiffres).

❖ Remplacement de sous-chaînes

`${paramètre/mod/ch}` : **bash** recherche dans la valeur de ***paramètre*** la plus longue sous-chaîne satisfaisant le modèle ***mod*** puis remplace cette sous-chaîne par la chaîne ***ch***. Seule la première sous-chaîne trouvée est remplacée. La valeur de ***paramètre*** n'est pas modifiée. Caractères et expressions génériques peuvent être présents dans ***mod***.

Ce mécanisme de remplacement comprend plusieurs aspects :

(1) Remplacement de la première occurrence

Ex : `$ v=totito`

```
$ echo ${v/to/lo}
```

```
lotito
```

```
$
```

La valeur de la variable *v* (***totito***) contient deux occurrences du modèle *to*. Seule la première occurrence est remplacée par la chaîne *lo*.



```

Ex : $ set topo
      $ echo $1
      topo
      $
      $ echo ${1/o/i}
      tipo
      $

```

(2) Remplacement de la plus longue sous-chaîne

```

Ex : $ v=abcfefg
      $ v1=${v/b*f/toto} => utilisation du caractère générique *
      $ echo $v1
      atotog
      $

```

Deux sous-chaînes de *v* satisfont le modèle *b*f* : *bcf* et *bcfef*

C'est la plus longue qui est remplacée par *toto*.

\${paramètre//mod/ch} : contrairement à la syntaxe précédente, toutes les occurrences (et non seulement la première) satisfaisant le modèle **mod** sont remplacées par la chaîne **ch**

```

Ex : $ var=tobatoba
      $ echo ${var//to/tou}
      toubatouba
      $
      $ set topo
      $ echo $1
      topo
      $
      $ echo ${1//o/i}
      tipi
      $

```

\${paramètre//mod/} : lorsque la chaîne **ch** est absente, la première ou toutes les occurrences (suivant la syntaxe utilisée) sont supprimées

```

Ex : $ v=123azer45ty
      $ shopt -s extglob
      $ echo ${v//+([[:lower:]])/}
      12345
      $

```

L'expression générique **+([[:lower:]])** désigne la plus longue suite non vide de minuscules. La syntaxe utilisée signifie que toutes les occurrences doivent être traitées : la variable *v* contient deux occurrences (**azer** et **ty**). Le traitement à effectuer est la suppression.



Il est possible de préciser si l'on souhaite l'occurrence cherchée en début de chaîne de **paramètre** (syntaxe à utiliser : **#mod**) ou bien en fin de chaîne (**%mod**).

```
Ex : $ v=automoto
      $ echo ${v/#aut/vel}
      velomoto
      $
      $ v=automoto
      $ echo ${v/%to/teur}
      automoteur
      $
```

6.6. DECLARATION ET VISIBILITE DES FONCTIONS.

➤ Définition d'une fonction

Comme les << vrais >> langages de programmation, Bash supporte les fonctions, bien qu'il s'agisse d'une implémentation quelque peu limitée. Une fonction est une sous-routine, un [bloc de code](#) qui implémente un ensemble d'opérations, une << boîte noire >> qui réalise une tâche spécifiée. Quand il y a un code répétitif, lorsqu'une tâche se répète avec quelques légères variations, alors utilisez une fonction.

```
function nom_fonction {
commande...
}
```

ou

```
nom_fonction () {
commande...
}
```

Cette deuxième forme plaira aux programmeurs C (et est plus portable).

Comme en C, l'accolade ouvrante de la fonction peut apparaître de manière optionnelle sur la deuxième ligne.

```
nom_fonction ()
{
commande...
}
```

Les fonctions sont appelées, *lancées*, simplement en invoquant leur noms.



Exemple : fonction simple

```
#!/bin/bash

funky ()
{
    echo "Ceci est la fonction funky."
    echo "Maintenant, sortie de la fonction funky."
} # La déclaration de la fonction doit précéder son appel.

# Maintenant, appelons la fonction.

funky

exit 0
```

La définition de la fonction doit précéder son premier appel. Il n'existe pas de méthode pour << déclarer >> la fonction, comme, par exemple, en C.

```
f1
# Donnera un message d'erreur, car la fonction "f1" n'est pas encore définie.

declare -f f1    # Ceci ne nous aidera pas plus.
f1              # Toujours un message d'erreur.

# Néanmoins...

f1 ()
{
    echo "Appeler la fonction \"f2\" à partir de la fonction \"f1\"."
    f2
}

f2 ()
{
    echo "Fonction \"f2\"."
}

f1 # La fonction "f2" n'est pas appelée jusqu'à ce point, bien qu'il soit
   # référencé avant sa définition.
   # C'est autorisé.

# Merci, S.C.
```

Il est même possible d'intégrer une fonction dans une autre fonction bien que cela ne soit pas très utile.




```

f1 ()
{

    f2 () # intégrée
    {
        echo "La fonction \"f2\", à l'intérieur de \"f1\"."
    }

}

f2 # Donne un message d'erreur.
    # Même un "declare -f f2" un peu avant ne changerait rien.
echo

f1 # Ne donne rien, car appeler "f1" n'appelle pas automatiquement "f2".
f2 # Maintenant, il est tout à fait correct d'appeler "f2",
    # car sa définition est visible en appelant "f1".

# Merci, S.C.

```

Les déclarations des fonctions peuvent apparaître dans des endroits bien étonnants, même là où irait plutôt une commande.

```

ls -l | foo() { echo "foo"; } # Autorisé, mais sans intérêt.

if [ "$USER" = bozo ]
then
    bozo_salutations () # Définition de fonction intégrée dans une construction if/then.
    {
        echo "Bonjour, Bozo."
    }
fi

bozo_salutations      # Fonctionne seulement pour Bozo, et les autres utilisateurs ont une erreur.

# Quelque chose comme ceci peut être utile dans certains contextes.
NO_EXIT=1 # Active la définition de fonction ci-dessous.

[[ $NO_EXIT -eq 1 ]] && exit() { true; } # Définition de fonction dans une "liste et".
# Si $NO_EXIT vaut 1, déclare "exit ()".
# Ceci désactive la commande intégrée "exit" en créant un alias vers "true".

exit # Appelle la fonction "exit ()", et non pas la commande intégrée "exit".

# Merci, S.C.

```



En Shell les fonctions se comportent comme des variables, elles ont donc une visibilité locale ou globale

7. POSTAMBULE ET RETOUR DE SHELL-SCRIPT

7.1. SORTIE DU SHELL-SCRIPT

Un script Shell peut être vu comme une suite de commandes à laquelle on a donné un nom. Il s'en suit que le code de retour d'un programme Shell est le code de retour de la dernière commande qu'il a exécutée.

Soit le programme Shell lvi contenant l'unique commande ls vi.. Après exécution, le code de retour sera celui de la commande ls vi (dernière commande exécutée).

```
$ cat lvi
#!/bin/sh
ls vi
```

```
$ lvi
ls: vi: Aucun fichier ou répertoire de ce type
```

```
$ echo $?
1 => code de retour de la dernière commande exécutée par lvi
```

Prenons un autre exemple. Le fichier lvi1 contient deux commandes ls vi et echo Fin :

```
$ cat lvi1
#!/bin/sh
ls vi
echo Fin
```

```
$ lvi1
ls: vi: Aucun fichier ou répertoire de ce type
Fin
```

```
$ echo $?
0 => code de retour de la dernière commande exécutée par lvi1 (echo Fin)
```

Il est parfois nécessaire de positionner explicitement le code de retour d'un programme Shell avant qu'il ne se termine pour signaler une erreur particulière à l'utilisateur : on utilise alors la commande interne **exit**.



7.2. FONCTION DE SORTIE.

Sa syntaxe est particulièrement simple : **exit** [*n*]

Elle provoque l'arrêt du programme Shell avec un code de retour égal à *n*. Si *n* n'est pas précisé, le code de retour fourni est celui de la dernière commande exécutée.

```
$ cat lvi2
#!/bin/sh
ls vi
exit 23
```

```
$ lvi2
ls: vi: Aucun fichier ou répertoire de ce type
```

```
$ echo $?
23 => code de retour de exit 23
```

Le programme Shell lvi2 positionne un code de retour différent (ici égal à 23) après exécution de la commande ls vi.

7.3. CONVENTIONS UTILISEES

Les exercices sont fait sur le Shell ,a adapter au Shell de son système

7.4. - VALEUR DE RETOUR.

On ne doit pas confondre le résultat d'une commande et son code de retour : le résultat correspond à ce qui est écrit sur sa sortie standard. Le code de retour indique uniquement si l'exécution de la commande s'est bien effectuée ou non. Parfois, on est intéressé uniquement par le code de retour d'une commande et non par les résultats qu'elle produit sur la sortie standard ou la sortie standard pour les messages d'erreurs.

Dans l'exemple ci-dessous, sortie standard et sortie standard pour les messages d'erreur de la commande grep ont été redirigées vers la « poubelle » /dev/null.

```
$ grep toto /etc/passwd > /dev/null 2>&1
$
```

```
$ echo $?
1 => on en déduit que la chaîne toto n'est pas présente dans /etc/passwd
```



7.5. ENCHAINEMENT DE SHELL-SCRIPT

Il est possible de lancer une chaîne de plusieurs shell-scripts sur une même ligne de commande il suffit de les séparer par un « ; ».

Attention au chemin d'exécution.

`./file1 ; ./file2 ; ...../filen ; entrée`

- Commande simple (en premier plan)
 - o `$ commande [ arg ... ]`
- Commande simple (en arrière plan)
 - o `$ commande [ arg ... ] &`
- Pipeline (tube de communication)
 - o `$ commande1 [ arg ... ] | commande2 [ arg ... ] | ...`
- Séquencement de commandes (en premier plan)
 - o `$ commande1 [ arg ... ]; commande2 [ arg ... ]; commande3 ...`
- Exécution par un nouveau shell ()
 - o `$ sh commande [ arg ... ]`
 - o `$ (commande [ arg ... ])`
 - o `$ (commande [ arg ... ] &`
- Regroupement de commandes
 - o `$ (com1 [ arg ... ]; com2 [ arg ... ]; com3 ...)`
 - o `$ (com1 [ arg ... ]; com2 [ arg ... ]; com3 ...)&`

8. STRUCTURES DE CONTROLE DU SHELL

8.1. COMMANDES SIMPLES, PIPELINES, ET LISTES DE PIPELINES.

Pipelines, Listes

- Pipeline
 - o un pipeline est une séquence d'une ou plusieurs commandes séparées par le caractère `|`.
 - o format : `[!] command1 [ | command2 ... ]`
 - o la sortie standard de la première commande est connectée à l'entrée standard de la commande suivante, et ainsi de suite.
- Liste
 - o une liste est une séquence d'un ou plusieurs pipelines séparés par un des opérateurs : `;;`, `&`, `&&`, `||`.
 - o une liste est optionnellement terminée par un `;;`, `&`, `<newline>`.
 - o précédences des opérateurs: `&&` > `||` > `;` = `&`



si une commande est terminée par l'opérateur **&**, le shell exécute la commande en arrière plan (*background*) dans un sous-shell.

8.2. COMMANDES COMPOSEES, SOUS-SHELLS ET FONCTIONS.

L'exécution d'un script Shell lance une nouvelle instance de l'interpréteur de commande. De la même manière que sont interprétées les commandes tapées en ligne de commande, un script bash exécute une liste de commandes lues dans un fichier. Chaque script Shell exécuté est en réalité un sous-processus du Shell [parent](#), celui qui vous donne une invite à la console ou dans une fenêtre xterm

Un script Shell peut également lancer des sous-processus. Ces *sous-Shell*s permettent au script de faire de l'exécution en parallèle, donc d'exécuter différentes tâches simultanément.

En général, une [commande externe](#) dans un script [lance](#) un sous-processus alors qu'une [commande intégrée](#) Bash ne le fait pas. Pour cette raison, les commandes intégrées s'exécutent plus rapidement que leur commande externe équivalente.

Liste de commandes entre parenthèses

(commande1; commande2; commande3; ...)

Une liste de commandes placées entre *parenthèses* est exécutée sous forme de sous-Shell

Les variables utilisées dans un sous Shell *ne sont pas* visibles en dehors du code du sous-Shell. Elles ne sont pas utilisables par le [processus parent](#), le Shell qui a lancé le sous-Shell. Elles sont en réalité des [variables locales](#).



Exemple : Etendue des variables dans un sous-Shell

```
#!/bin/bash
# subShell.sh

echo

variable_externe=Outer

(
variable_interne=Inner
echo "A partir du sous-Shell, \"variable_interne\" = $variable_interne"
echo "A partir du sous-Shell, \"externe\" = $variable_externe"
)

echo

if [ -z "$variable_interne" ]
then
echo "variable_interne non défini dans le corps principal du Shell"
else
echo "variable_interne défini dans le corps principal du Shell"
fi

echo "A partir du code principal du Shell, \"variable_interne\" = $variable_interne"
# $variable_interne s'affichera comme non initialisé parce que les variables
# définies dans un sous-Shell sont des "variables locales".

echo

exit 0
```

8.3. MECANISMES DE SELECTION ET D'ITERATION.

8.3.1. ITERATION WHILE

La commande interne **while** correspond à l'itération *tant que* présente dans de nombreux langages de programmation.

Syntaxe : **while** *suite_cmd1*
 do
 suite_cmd2
 done

La suite de commandes *suite_cmd1* est exécutée; si **son code de retour est égal à 0**, alors la suite de commandes *suite_cmd2* est exécutée, puis *suite_cmd1* est re-exécutée. Si son code de retour est différent de 0, alors l'itération se termine.



En d'autres termes, *suite_cmd2* est exécutée autant de fois que le code de retour de *suite_cmd1* est égal à zéro.

L'originalité de cette structure de contrôle est que le test ne porte pas sur une condition booléenne (vraie ou fausse) mais sur le code de retour issu de l'exécution d'une suite de commandes.

En tant que mot-clés, **while**, **do** et **done** doivent être les premiers mots d'une commande.

Une commande **while**, comme toute commande interne, peut être écrite directement sur la ligne de commande.

```
Ex: $ while who | grep logware >/dev/null
      > do
      > echo "l'utilisateur logware est encore connecte"
      > sleep 5
      > done
```

l'utilisateur logware est encore connecte

Le fonctionnement est le suivant : la suite de commandes *who | grep logware* est exécutée. Son code de retour sera égal à 0 si le mot *logware* est présent dans les résultats engendrés par l'exécution de la commande unix **who**, c'est à dire si l'utilisateur *logware* est connecté. Dans ce cas, la ou les lignes correspondant à cet utilisateur seront affichées sur la sortie standard de la commande **grep**. Comme seul le code de retour est intéressant et non le résultat, la sortie standard de **grep** est redirigée vers le puits (*/dev/null*). Enfin, le message est affiché et le programme « s'endort » pendant 5 secondes. Ensuite, la suite de commandes *who | grep logware* est ré exécutée. Si l'utilisateur s'est totalement déconnecté, la commande **grep** ne trouve aucune ligne contenant la chaîne *logware*, son code de retour sera égal à 1 et le programme sortira de l'itération.

Commandes internes while et deux-points

La commande interne **deux-points** associée à une itération **while** compose rapidement un serveur (démon) rudimentaire.

```
Ex : $ while : => boucle infinie
      > do
      > who | cut -d' ' -f1 >>fic => traitement à effectuer
      > sleep 300 => temporisation
      > done &
      [1] 12568
      $
```



Lecture du contenu d'un fichier texte

La commande interne **while** est parfois utilisée pour lire le contenu d'un fichier texte. La lecture s'effectue alors ligne par ligne. Il suffit pour cela :

- de placer une commande interne **read** dans *suite_cmd1*
- de placer les commandes de traitement de la ligne courante dans *suite_cmd2*
- de rediriger l'entrée standard de la commande **while** avec le fichier à lire.

Syntaxe : **while read** [*var1 ...*]

do

commande(s) de traitement de la ligne courante

done < *fichier_à_lire*

Ex : programme **wh** qui affiche les noms des utilisateurs connectés

```
#!/bin/bash
# @(#) wh
who > tmp
while read nom reste
do
  cho $nom
done < tmp
rm tmp
```

Exercice : On dispose d'un fichier *personnes* dont chaque ligne est constituée du prénom et du genre (*m* pour masculin, *f* pour féminin) d'un individu.

```
Ex : $ cat personnes
arthur m
pierre m
dominique f
paule f
sylvie f
jean m
$
```

Ecrire un programme Shell **tripersonnes** qui crée à partir de ce fichier, un fichier *garcons* contenant uniquement les prénoms des garçons et un fichier *filles* contenant les prénoms des filles.

```
Ex : $ tripersonnes
$
$ cat filles
dominique
paule
sylvie
$
$ cat garcons
arthur
pierre
jean
$
```



Lorsque le fichier à lire est créé par une commande *cmd*, comme dans le programme **wh**, on peut utiliser la syntaxe :

```
cmd | while read [ var1 ... ]
do
    commande(s) de traitement de la ligne courante
done
```

Ex : wh1

```
#!/bin/bash
# @(#) wh1
who | while read nom reste
do
    echo $nom
done
```

Par rapport au programme Shell **wh**, il est inutile de gérer le fichier temporaire *tmp*.

Exercice : En utilisant la commande *ruptime* écrire un programme Shell *up* qui affiche le nom de toutes les machines qui sont dans l'état *up*

Exercice :

- Écrire un programme Shell *etat1* prenant un nom de machine en argument et qui affiche son état (*up* ou *down*). On ne considère aucun cas d'erreur.
- Modifier ce programme, soit *etat2*, pour qu'il affiche un message d'erreur lorsque le nom de la machine passé en argument n'est pas visible à l'aide de la commande **ruptime**.
- Modifier le programme précédent, soit *etat*, pour qu'il vérifie également le nombre d'arguments passés lors de l'appel.

8.3.2. ITERATION FOR

L'itération **for** possède plusieurs syntaxes dont les deux plus générales sont :

Première forme : **for** *var*

```
do
    suite_de_commandes
done
```

Lorsque cette syntaxe est utilisée, la variable *var* prend successivement la valeur de chaque paramètre de position initialisé.

Ex : programme **for_arg**

```
-----
#!/bin/bash
for i
do
    echo $i
    o "Passage a l'argument suivant ..."
done
-----
```



Ex : \$ **for_arg** un deux => deux paramètres de position initialisés

```
un
Passage a l'argument suivant ...
deux
Passage a l'argument suivant ...
$
```

Exercice : En utilisant la commande **ruptime**, écrire un programme Shell **etatgene** prenant en arguments un ou plusieurs noms de machines et affiche pour chacune d'elles leur état (*up* ou *down*).

La commande composée **for** traite les paramètres de position sans tenir compte de la manière dont ils ont été initialisés (lors de l'appel d'un programme Shell ou bien par la commande interne **set**).

Ex: programme **for_set**

```
-----
#!/bin/bash
set $(date)
for i
do
echo $i
done
-----
```

Ex : \$ **for_set**

```
dimanche
17
décembre
2006,
11:22:10
(UTC+0100)
$
```

Deuxième forme : **for var in liste_mots**

```
do
suites_de_commandes
done
```

La variable *var* prend successivement la valeur de chaque mot de *liste_mots*.

Ex : programme **for_liste**

```
-----
#!/bin/bash
for a in toto tata
do
echo $a
done
-----
```



Ex : \$ **for_liste**

```
toto
tata
$
```

Si *liste_mots* contient des substitutions, elles sont préalablement traitées par **bash**.

Ex: programme **affich.ls**

```
-----
#!/bin/bash
for i in tmp $(pwd)
do
echo " --- $i ---"
ls $i
done
-----
```

Ex : \$ **affich.ls**

```
--- tmp ---
gamma
--- /home/logware/Rep ---
affich.ls alpha beta tmp
$
```

Exercice : Ecrire un programme Shell **lsrep** ne prenant aucun argument, qui demande à l'utilisateur de saisir une suite de noms de répertoires et qui affiche leur contenu respectif.

8.3.3. CHOIX IF

La commande interne **if** implante le choix alternatif.

Syntaxe : **if** *suite_de_commandes1*

```
then
suite_de_commandes2
[ elif suite_de_commandes ; then suite_de_commandes ] ...
[ else suite_de_commandes ]
fi
```

Le fonctionnement est le suivant : *suite_de_commandes1* est exécutée ; si son code de retour est égal à **0**, alors *suite_de_commandes2* est exécutée sinon c'est la branche **elif** ou la branche **else** qui est exécutée, si elle existe.

Ex : programme **rm1**

```
-----
#!/bin/bash
if rm $1 2>/dev/null
then echo $1 a ete supprime
else echo $1 n'a pas ete supprime
```



fi

Ex : \$ >toto => création du fichier *toto*

```
$  
$ rm1 toto  
toto a ete supprime  
$  
$ rm1 toto  
toto n'a pas ete supprime  
$
```

Lorsque la commande *rm1 toto* est exécutée, si le fichier *toto* est effaçable, le fichier est effectivement supprimé, la commande unix **rm** renvoie un code de retour égal à **0** et c'est la suite de commandes qui suit le mot-clé **then** qui est exécutée ; le message *toto a ete supprime* est affiché sur la sortie standard.

Si *toto* n'est pas effaçable, l'exécution de la commande **rm** échoue ; celle-ci affiche un message sur la sortie standard pour les messages d'erreur que l'utilisateur ne voit pas car celle-ci a été redirigée vers le puits, puis renvoie un code de retour différent de 0. C'est la suite de commandes qui suit **else** qui est exécutée : le message *toto n'a pas ete supprime* s'affiche sur la sortie standard.

Les mots **if**, **then**, **else**, **elif** et **fi** sont des mots-clé. Par conséquent, pour indenter une structure **if** suivant le « style langage C », on pourra l'écrire de la manière suivante :

```
if suite_de_commandes1 ; then  
suite_de_commandes2  
else  
suite_de_commandes ]  
fi
```

La structure de contrôle doit comporter autant de mots-clés **fi** que de **if** (une branche **elif** ne doit pas se terminer par un **fi**).

Ex : **if** ... **if** ...
then ... **then** ...
elif ... **else if** ...
then ... **then** ...
fi fi
fi

Dans une succession de **if** imbriqués où le nombre de **else** est inférieur au nombre de **then**, le mot-clé **fi** précise l'association entre les **else** et les **if**.



Ex : if ...
then ...
if ...
then ...
fi
else ...
fi

8.4. MENUS

8.4.1. CHOIX MULTIPLE CASE

Syntaxe : **case** *mot* in
 [*modèle* [| *modèle*] ...] *suite_de_commandes* ;;] ...
esac

Le Shell évalue la valeur de **mot** puis compare séquentiellement cette valeur à chaque modèle.

Dès qu'un modèle correspond à la valeur de **mot**, la suite de commandes associée est exécutée, terminant l'exécution de la commande interne composée **case**.

Les mots **case** et **esac** sont des mots-clé ce qui signifie que chacun d'eux doit être le premier mot d'une commande.

suite_de_commandes doit se terminer par deux caractères point-virgule collés, de manière à ce qu'il n'y ait pas d'ambiguïté avec l'enchaînement séquentiel de commandes *cmd1* ; *cmd2*.

Un *modèle* peut être construit à l'aide des caractères et expressions génériques de **bash** [cf. § *Caractères et expressions génériques*]. Dans ce contexte, le symbole | signifie OU.

Pour indiquer le cas par défaut, on utilise le modèle *. Ce modèle doit être placé à la fin de la structure de contrôle **case**.

Le code de retour de la commande composée **case** est celui de la dernière commande exécutée de *suite_de_commandes*.

Ex 1 : programme Shell **oui** affichant *OUI* si l'utilisateur a saisi le caractère *o* ou *O*

```
#!/bin/bash
# @(#) oui
read -p "Entrez votre réponse : " rep
case $rep in
o|O ) echo OUI ;;
*) echo Indéfini
esac
```



Rq : il n'est pas obligatoire de terminer par `;;` la dernière *suite_de_commandes*

Ex 2 : programme Shell **nombre** prenant une chaîne de caractères en argument et qui affiche cette chaîne si elle est constituée d'une suite de chiffres

```
#!/bin/bash
# @(#) nombre
t-extglob
shop s case$1in
[:digit:])) ) echo "$1 est une suite de chiffres" ;;
+([esac
```

Exercice : Ecrire un programme Shell **4arg** qui vérifie que le nombre d'arguments passés lors de l'appel du programme est égal à 4 et écrit suivant le cas le message *"Correct"* ou le message *"Erreur"*.

Exercice : Ecrire un programme Shell **reconnaitre** qui demande à l'utilisateur d'entrer un mot, puis suivant le premier caractère de ce mot, indique s'il commence par un chiffre, une minuscule, une majuscule ou une autre sorte de caractère.

Exercice : En utilisant la commande unix **date** et la locale standard [cf. *Substitution de commandes*], écrire un programme **datefr** qui affiche la date courante de la manière

9. COMMANDES INTERNES ET EXTERNES

Commandes internes

Les commandes internes au Shell font partie intégrante du Shell et leur exécution n'implique pas la création d'un nouveau processus. Exemples : `pwd`, `cd`.

Commandes externes

Ce sont des programmes binaires généralement situés dans le répertoire `/bin` que le Shell va exécuter sous la forme d'un nouveau processus. Exemples : `ls`, `gzip`.

9.1. ENTREES/SORTIES

Entrée standard

L'entrée standard est le canal d'entrée des données qui est utilisé par le système. Par défaut c'est le clavier. Ainsi, les commandes du Shell prennent leur paramètres sur l'entrée standard.



Sortie standard

La sortie standard est le canal de sorties des données. C'est par ce canal que transitent les données résultant de l'exécution d'une commande. C'est en général un terminal, c'est-à-dire l'écran. Ainsi, les commandes du Shell écrivent très souvent des résultats sur la sortie standard.

Sortie d'erreur standard

La sortie d'erreur standard est le canal par lequel les messages d'erreurs transitent, c'est en général l'écran. Il arrive quelque fois qu'une fenêtre soit spécialement dédiée à ce canal.

Dès qu'un code d'erreur est généré par une commande, il est envoyé un message sur ce canal.

Redirections

Il est possible de changer temporairement les entrées et sorties standard lors de l'exécution d'une commande.

Par exemple on souhaite écrire dans un fichier la liste des fichiers d'un répertoire. La commande `ls` permet de lister les fichiers d'un répertoire. Cette commande envoie le résultat de sa recherche sur la sortie standard (écran).

Ex:

```
$ ls
amoi.c      montage.jpg  tp3.c
lettre.doc  tp1.c       zizitop.mp3
monprog.c   tp2.c
```

Pour rediriger la sortie standard sur un fichier, on utilise le caractère spécial `>`.

Ex:

```
$ ls > liste.txt
```

Si on affiche à l'écran le contenu du fichier, on verra qu'il contient ce que la commande aurait du afficher à l'écran.

Ex:

```
$ cat liste.txt
amoi.c      montage.jpg  tp3.c
lettre.doc  tp1.c       zizitop.mp3
monprog.c   tp2.c
```



Le caractère > permet de créer le fichier si celui-ci n'existe pas lors de l'exécution de la commande. Si le fichier existe déjà, son contenu est écrasé.

Pour conserver le contenu du fichier intact et écrire à sa suite, on utilise le caractère spécial >>.

Ex:

```
echo "Liste de mon répertoire" >> liste.txt
```

La commande echo permet d'afficher du texte sur la sortie standard qui est ici redirigée vers le fichier *liste.txt* à la suite duquel on écrit la chaîne de caractères passée en argument.

Ex:

```
$ cat liste.txt
amoi.c      montage.jpg  tp3.c
lettre.doc  tp1.c      zizitop.mp3
monprog.c   tp2.c
Liste de mon répertoire
```

9.2. INTERACTIONS AVEC LE SYSTEME

Le répertoire courant est propre à chaque processus (et hérité d'un processus père vers ses processus fils).

Un processus peut changer de répertoire courant par l'appel système chdir.

Un Shell interface cet appel système par la commande cd (qui est donc nécessairement builtin = cablée dans le Shell).

```
man execve
man chdir
man fork
```

9.3. ARGUMENTS EN LIGNE DE COMMANDE

Il est possible d'exécuter un script en lui passant des arguments comme pour n'importe quelle autre commande.

Le tableau suivant résume les variables accessibles à un script :



Variable	Description
\$#	Nombre d'arguments.
\$*	Liste des arguments.
\$0	Nom de la commande.
\$1	Valeur du premier paramètre.
\$i	Valeur du ième paramètre si i compris entre 1 et 9.
\$9	Valeur du neuvième paramètre.

9.4. OPERATIONS DE TESTS

Les conditions des structures peuvent être évaluées grâce à la commande test.

Chaînes de caractères

test -z *chaine*

Renvoie vrai si *chaine* est vide.

test -n *chaine*

Renvoie vrai si *chaine* n'est pas vide.

test *chaine_1* = *chaine_2*

Renvoie vrai si les deux chaînes sont égales.

test *chaine_1* != *chaine_2*

Renvoie vrai si les deux chaînes ne sont pas égales.

Chaînes numériques

test *chaine_1* *opérateur* *chaine_2*

Renvoie vrai si les chaînes *chaine_1* et *chaine_2* sont dans l'ordre relationnel défini par l'*opérateur* qui peut être l'un parmi ceux du tableau ci-après.



Opérateur	Signification
-eq	=
-ne	<>
-lt	<
-le	<=
-gt	>
-ge	>=

Fichiers

test condition fichier

Renvoie vrai si le *fichier* vérifie la *condition* qui peut être l'une parmi celles décrites dans le tableau ci-après.

Opérateur	Renvoie vrai si
-p	c'est un tube nommé
-f	fichier ordinaire
-d	répertoire
-c	fichier spécial en mode caractère
-b	fichier spécial en mode bloc
-r	accès en lecture
-w	accès en écriture
-x	accès en exécution
-s	fichier non vide

Autres opérateurs

Il est possible de combiner des tests grâce au parenthésage et à l'utilisation des opérateurs booléens du tableau suivant :



Opérateur	Signification
!	négation
-a	conjonction (et)
-o	disjonction (ou)

10. MECANISMES COMPLEMENTAIRES / DEBUGGING D'UN SHELL-SCRIPT

10.1. COMMANDES DE DEBUGGING.

Débugger un script Shell peut être gênant (lecture du code). Il ya plusieurs façons de procéder au débugging d'un script.

L'option -x debugge un script Shell

Lancer le Shell avec l'option -x

```
$ bash -x script-name
```

```
$ bash -x domains.sh
```

Il existe également des commandes internes

Bash Shell offers debugging options which can be turn on or off using set command.

Le Shell bash divers options de débuggage qui peuvent être actives avec la commande et

=> set -x : → Affiche une commande et ses arguments pendant l'exécution.

=> set -v : → Affiche les entrées du Shell pendant la lecture .

Les commandes ci dessus dans la script lui-même



```
#!/bin/bash
clear
# activé le debug
set -x
for f in *
do
    file $f
done
# desactive l'option debug
set +x
ls
```

On peut aussi remplacer

```
#!/bin/bash
with (pour le debug)
#!/bin/bash -xv
```

10.2. SIGNAUX DE TRACE & JOURNALISATION.

Trace des appels aux fonctions

Le tableau prédéfini **FUNCNAME** contient le nom des fonctions en cours d'exécution, matérialisant la pile des appels.

Le programme Shell *traceAppels* affiche le contenu de ce tableau au début de son exécution, c'est-à-dire hors de toute fonction : le contenu du tableau **FUNCNAME** est vide (a). Puis la fonction *f1* est appelée, **FUNCNAME** contient les noms *f1* et *main* (b). La fonction *f2* est appelée par *f1* : les valeurs du tableau sont *f2*, *f1* et *main* (c).

Lorsque l'on est à l'intérieur d'une fonction, la syntaxe **\$FUNCNAME** (ou **\${FUNCNAME[0]}**) renvoie le nom de la fonction courante.

Trace Appels

```
-----
#!/bin/bash
function f2
{
    echo " ----- Dans f2 : "
    echo " ----- FUNCNAME : $FUNCNAME"
    echo " ----- tableau FUNCNAME[] : ${FUNCNAME[*]}"
}
function f1
{
    echo " --- Dans f1 : "
    echo " --- FUNCNAME : $FUNCNAME"
    echo " --- tableau FUNCNAME[] : ${FUNCNAME[*]}"
    echo " --- Appel a f2 "
```



```

echo
f2
}
echo "Debut :"
echo "FUNCNAME : $FUNCNAME"
echo "tableau FUNCNAME[] : ${FUNCNAME[*]}"
echo
f1

```

Ex : \$ **traceAppels**

```

Debut :
FUNCNAME :
tableau FUNCNAME[] : (a)
--- FUNCNAME : f1
--- tableau FUNCNAME[] : f1 main (b)
--- - Appel a f2
----- Dans f2 :
----- FUNCNAME : f2
----- tableau FUNCNAME[] : f2 f1 main

```

11. ROBUSTESSE D'UN SHELL-SCRIPT

11.1. VERIFIER L'INITIALISATION DES VARIABLES

Avec les fonctions eval et test toujours vérifier les arguments .

11.2. GESTION AVANCEE DES ARGUMENTS EN LIGNE DE COMMANDE (GETOPTS)

GETOPTS

L'analyse des arguments d'une commande permet de :

- détecter les options que l'utilisateur veut voir appliquer,
- récupérer les valeurs, noms de fichiers, de répertoire ...
- détecter des erreurs de l'utilisateur, que l'on doit identifier et expliquer clairement.

Si le programmeur veut simplifier cette analyse, il imposera une syntaxe rigide à l'appel de son script, ce qui ne le rend pas très attrayant.

Si, au contraire il souhaite apporter de la souplesse à l'appel du script, les diverses possibilités de présentation des options entraînent, si on **utilise seulement** les instructions 'if then ... elif ... fi' ou 'case ... esac', une suite d'instructions tellement complexe que *ça marche une fois* et on perd un temps fou à mettre au point sérieusement ces lignes de code.



Un outil existe, pour faciliter ce travail c'est la commande *interne* **getopts** pour les interprètes de commande (Shell: bash, ksh ...), ou la fonction **getopt** pour les langages C ou C++, ou la classe **GetOpt** ... pour Java, Perl ... Nous présentons ci-après un exemple d'analyse, programmé pour l'interprète de commande ksh ou bash.

getopts sert à analyser les paramètres passés au script.

Ex:

```
while getopts ad:h parm
do
case $parm in
a) all=true;;
d) dest_dir=$OPTARG;;
h) echo ...
echo...
;;
?) echo usage
exit 1;;
esac
done
shift $((OPTARG-1)) #OPTARG est un pointeur sur la première chaîne de texte qui n'est
pas de la forme « -x »
```

Remarque

getopts est une commande interne de bash. La chaîne ad:h permet de dire que l'on permet les options -a -d [argument] et -h.

12. AUTRES POINTS

12.1. CAS PARTICULIER D'EXECUTION D'UN SHELL-SCRIPT PAR CRON

Crontab est un utilitaire incontournable qui permet de programmer des tâches régulières (/répétées) de façon simple, sur base d'un fichier texte qui, sous Debian se situe ici: /var/spool/cron/crontabs/

Notez donc que cela peut varier d'une distrib à une autre.



Il y a un fichier crontab par utilisateur, et c'est bien naturel : ce que veut préprogrammer l'un n'est pas la même chose que ce que voudrait préprogrammer un autre utilisateur.

Amusez-vous par ex. à programmer crontab pour que Saytime vous dise l'heure à des moments précis de la journée, ou pour exécuter un updatedb une fois par jour, etc.

La structure du fichier crontab se compose d'une ligne par tâche :

- Les cinq premiers champs déterminent la fréquence d'exécution d'une tâche,
- Les autres sont simplement la commande à exécuter.

Les cinq premiers champs sont:

minute (0-59)

heure (0-23)

jour du mois (1-31)

mois de l'année (1-12)

jour de la semaine (0-6, dimanche = 0)

On édite le fichier "perso" crontab de la façon suivante:

crontab -e 'enter'

i (pour entrer dans le mode "insert" de Vi/ViM)

vous pouvez maintenant entrer vos paramètres+la commande souhaitée

ESC (pour clôturer le mode "insert")

:wq 'enter' ("w)rite" and "q)uit"), pour sauver et quitter Vi/ViM

Voilà, vous retrouvez le prompt de la console!

Pour vérifier que ce que vous venez d'entrer a bien été pris en compte : crontab -l 'enter'

Pour supprimer crontab: crontab -r 'enter'

Ex:

Exécute 'commande' le 1er de chaque mois à 20h30

30 20 1 * * 'commande' >/dev/null 2>&1

par exemple:

30 20 1 * * /usr/bin/updatedb >/dev/null 2>&1

(si vous souhaitez être averti systématiquement par mail de l'exécution de la tâche, ne mettez pas >/dev/null 2>&1)

Exécute 'commande' tous les lundis à 23h30

30 23 * * 1 'commande' >/dev/null 2>&1



A noter que: "*" signifie 'pour toutes les occurrences'.

On peut également spécifier des séries pour chaque champ, par exemple:

1-5 signifie de 1 à 5 compris

1,3,5 signifie 1, 3 et 5

2,6-12,15,20-23 signifie 2 et 6 à 12 et 15 et 20 à 23

Ex:

Exécute script Shell chaque semaine, du lundi au vendredi, à 20h30

30 20 * * 1-5 'exemple.sh' >/dev/null 2>&1

Exécute un script Shell tous les jours à 10h15 et 22h15

15 10,22 * * * 'exemple.sh' >/dev/null 2>&1

IMPORTANT:

- ne jamais éditer crontab à la main, toujours utiliser crontab -e 'enter',
- utiliser crontab -l 'enter' pour lister les tâches enregistrées, et crontab -r 'enter' pour supprimer le fichier crontab actuel
- par défaut, cron redirige son standard error vers le mail de l'utilisateur.

Cela peut devenir gênant! Donc, pour éviter une inondation de mails liés à crontab, mieux vaut placer : 'notre commande' >/dev/null 2>&1 en sachant que "2>&1" signifie rediriger le standard error (2) vers le standard output (&1).

- Il faut toujours 5 entrées:

1 0 * * *

| | | | +- jour de la semaine

| | | +--- mois de l'année

| | +---- jour du mois

| +----- heure du jour

+----- minute

12.2. LA COMMANDE EVAL

Cette commande ordonne l'interprétation par le Shell de la chaîne passée en argument. On peut ainsi construire une chaîne que l'appel à **eval** permettra d'exécuter comme une commande !

Ex

message="Quelle est la date d'aujourd'hui ?

set \$message

echo \$# ---> le nombre de mots est 6

echo \$4 ---> affiche la chaîne "date"

eval \$4 ---> interprète la chaîne "date" comme une commande, donc ...



Il est souvent pratique de construire une chaine dont la valeur sera égale au libellé d'un enchainement de commandes (par ;).

Pour faire exécuter ces commandes contenues dans la chaine, on la passe comme argument de la commande **eval**

Ex

liste="date;who;pwd" (' ' ou " " obligatoires sinon le ; est un séparateur de commandes)

eval \$liste

---> exécute bien les 3 commandes

Ex

Soit la chaine \$user qui contient des information sur un compte à créer. S'il utilise un autre séparateur que ";" on fait appel à **tr** d'abord

```
user="login=toto ; mdp=moi ; nom='Monsieur Toto' ; groupe=profs"
```

```
eval $user
```

```
echo $login $mdp $nom $groupe
```

```
useradd -G $groupe $login
```

```
echo $mdp | (passwd --stdin $login)
```

13. EXTENSIONS DU KORN SHELL ET BASH

13.1. TABLEAUX DE VARIABLES. NOTATIONS SPECIFIQUES

Moins utilisés que les *chaînes de caractères* ou les *entiers*, **bash** intègre également les tableaux monodimensionnels.

13.1.1. DEFINITION ET INITIALISATION D'UN TABLEAU

Pour créer un tableau, on utilise généralement l'option **-a** (comme *array*) de la commande interne **declare** :

```
declare -a nomtab ...
```

Le tableau *nomtab* est simplement créé mais ne contient aucune valeur : le tableau est défini mais n'est pas initialisé.

Pour connaître les tableaux définis : **declare -a**

Ex : **\$ declare -a**

```
declare -a BASH_ARGC=()'
```



```

declare -a BASH_ARGV=()'
declare -a BASH_LINENO=()'
declare -a BASH_SOURCE=()'
declare -ar BASH_VERSINFO=('[0]="3" [1]="1" [2]="17" [3]="1" [4]="release" [5]="i486-pc-linux-gnu"')
declare -a DIRSTACK=()'
declare -a FUNCNAME=()'
declare -a GROUPS=()'
declare -a PIPESTATUS=('[0]="0"')
$

```

Pour définir et initialiser un tableau : **declare -a nomtab=(val0 val1 ...)**

Comme en langage C, l'indice d'un tableau débute toujours à **0** et sa valeur maximale est celle du plus grand entier positif représentable dans ce langage (**bash** a été écrit en C). L'indice peut être une expression arithmétique.

Pour désigner un élément d'un tableau, on utilise la syntaxe : **nomtab[indice]**

Ex : \$ **declare -a tab** => définition du tableau *tab*

```

$
$ read tab[1] tab[3]
coucou bonjour
$
$ tab[0]=hello
$

```

Il n'est pas obligatoire d'utiliser la commande interne **declare** pour créer un tableau, il suffit d'initialiser un de ses éléments :

Ex : \$ **array[3]=bonsoir** => création du tableau *array* avec
\$ => initialisation de l'élément d'indice 3

Trois autres syntaxes sont également utilisables pour initialiser globalement un tableau :

- **nomtab=(val0 val1 ...)**

- **nomtab=([indice]=val ...)**

Ex : \$ **arr=([1]=coucou [3]=hello)**
\$

- l'option **-a** de la commande interne **read** ou **readonly** :

Ex : \$ **read -a tabmess**
bonjour tout le monde
\$

13.1.2. VALEUR D'UN ELEMENT D'UN TABLEAU

On obtient la valeur d'un élément d'un tableau à l'aide la syntaxe : **\${nomtab[indice]}** **bash** calcule d'abord la valeur de l'indice puis l'élément du tableau est remplacé par sa valeur.



Il est possible d'utiliser toute expression arithmétique valide de la commande interne `((` pour calculer l'indice d'un élément.

```
Ex : $ echo ${tabmess[1]}
tout
$
$ echo ${tabmess[RANDOM%4]} # ou bien ${tabmess[${(RANDOM%4)}]}
monde
$
$ echo ${tabmess[1**2+1]}
le
$
```

Pour obtenir la longueur d'un élément d'un tableau : `${#nomtab[indice]}`

```
Ex : $ echo ${tabmess[0]}
bonjour
$
$ echo ${#tabmess[0]}
7 => longueur de la chaîne bonjour
$
```

Lorsqu'un tableau sans indice est présent dans une chaîne de caractères ou une expression, **bash** utilise l'élément d'indice **0**.

```
Ex : $ echo $tabmess
bonjour
$
```

Réciproquement, une variable non préalablement définie comme tableau peut être interprétée comme un tableau.

```
Ex : $ var=bonjour
$ echo ${var[0]} => var est interprétée comme un tableau à un seul élément
bonjour => d'indice 0
$ var=( coucou ${var[0]} )
$ echo ${var[1]} => var est devenu un véritable tableau
bonjour
$
```

Exercice : Ecrire un programme Shell **carte** qui affiche le nom d'une carte tirée au hasard d'un jeu de 32 cartes. On utilisera deux tableaux : un tableau *couleur* et un tableau *valeur*.

```
Ex : $ carte
      huit de carreau
$ carte
      as de pique
$
```



13.1.3. CARACTERISTIQUES D'UN TABLEAU

Le nombre d'éléments d'un tableau est désigné par : **`${#nomtab[*]}`**

Seuls les éléments initialisés sont comptés.

Ex : `$ echo ${#arr[*]}`

2

\$

Tous les éléments d'un tableau sont accessibles à l'aide de la notation : **`${nomtab[*]}`**

Seuls les éléments initialisés sont affichés.

Ex : `$ echo ${arr[*]}`

coucou hello

\$

Pour obtenir la liste de tous les indices conduisant à un élément défini d'un tableau, on utilise la syntaxe : **`${!nomtab[*]}`**

Ex : `$ arr=([1]=coucou bonjour [5]=hello)`

\$

`$ echo ${!arr[*]}`

1 2 5

\$

L'intérêt d'utiliser cette syntaxe est qu'elle permet de ne traiter que les éléments définis d'un tableau « à trous ».

Ex : `$ for i in ${!arr[*]}`

> **do**

> `echo "$i => ${arr[i]}"` => dans l'expression `${arr[i]}`, **bash** interprète

> **done** => directement *i* comme un entier

1 => coucou

2 => bonjour

5 => hello

\$

Pour ajouter un élément *val* à un tableau *tab* :

- à la fin : `tab[${#tab[*]}]=val`

- en tête : `tab=( val ${tab[*]} )`

Ex : `$ tab=( un deux trois )`

`$ echo ${#tab[*]}`

3

`$ tab[${#tab[*]}]=fin` => ajout en fin de tableau

`$ echo ${tab[*]}`

un deux trois fin

`$ tab=( debut ${tab[*]} )` => ajout en tête de tableau



```
$ echo ${tab[*]}
debut un deux trois fin
$
```

Exercice : Ecrire un programme Shell **distrib** qui crée un paquet de 5 cartes différentes tirées au hasard parmi un jeu de 32 cartes et affiche le contenu du paquet.

Exercice : Ecrire un programme Shell **tabnoms** qui place dans un tableau les noms de tous les utilisateurs enregistrés dans le système, affiche le nombre total d'utilisateurs enregistrés, puis tire au hasard un nom d'utilisateur.

Si l'on souhaite créer un tableau d'entiers on utilise la commande **declare -ai**. L'ordre des options n'a aucune importance.

```
Ex : $ declare -ai tabent=( 2 45 -2 )
$
```

Pour créer un tableau en lecture seule, on utilise les options **-ra** :

```
Ex : $ declare -ra tabconst=( bonjour coucou salut ) => tableau en lecture seule
$
$ tabconst[1]=ciao
-bash: tabconst: readonly variable
$
$ declare -air tabInt=( 34 56 ) => tableau (d'entiers) en lecture seule
$ echo ${tabInt[1] +10 }
66
$ (( tabInt[1] += 10 ))
-bash: tabInt: readonly variable
$
```

13.1.4.SUPPRESSION D'UN TABLEAU

Pour supprimer un tableau ou élément d'un tableau, on utilise la commande interne **unset**.

Suppression d'un tableau : **unset nomtab ...**

Suppression d'un élément d'un tableau : **unset nomtab[indice] ...**

13.2. OPERATIONS ARITHMETIQUES. LES ALIAS SUIVIS

13.2.1.OPERATEURS

Une expression arithmétique contenue dans une commande interne **((** peut être une valeur, un paramètre ou bien être construite avec un ou plusieurs opérateurs. Ces derniers proviennent du



langage C. La syntaxe des opérateurs, leur signification, leur ordre de priorité et les règles d'associativité s'inspirent également du langage C.

Les opérateurs traités par la commande **((** sont nombreux. C'est pourquoi, seuls les principaux opérateurs sont mentionnés dans le tableau ci-dessous, munis de leur associativité (**g** : de gauche à droite, **d** : de droite à gauche). Ne sont pas présentés dans ce tableau les opérateurs portant sur les représentations binaires (ex : **et bit à bit**, **ou bit à bit**, etc.).

Les opérateurs ayant même priorité sont regroupés dans une même classe et les différentes classes sont citées par ordre de priorité décroissante.

- (1) **a++ a--** post-incrémentation, post-décrémentation (**g**)
- (2) **++a --a** pré-incrémentation, pré-décrémentation (**d**)
- (3) **-a +a** moins unaire, plus unaire (**d**)
- (4) **! a** négation logique (**d**)
- (5) **a**b** exponentiation : a^b (**d**)
- (6) **a*b a/b a%b** multiplication, division entière, reste (**g**)
- (7) **a+b a-b** addition, soustraction (**g**)
- (8) **a<b a<=b a>b a>=b** comparaisons (**g**)
- (9) **a==b a!=b** égalité, différence (**g**)
- (10) **a&& b** ET logique (**g**)
- (11) **a| b** OU logique (**g**)
- (12) **a?b:c** opérateur conditionnel (**d**)
- (13) **a=b a*=b a%=b** opérateurs d'affectation (**d**)
a+=b a-=b
- (14) **a,b** opérateur virgule (**g**)

Ces opérateurs se décomposent en :

- opérateurs arithmétiques
- opérateurs d'affectations
- opérateurs relationnels
- opérateurs logiques
- opérateurs divers.

Opérateurs arithmétiques :

Les quatre opérateurs ci-dessous s'appliquent à une *variable*.

Post-incrémentation : **var++** la valeur de *var* est d'abord utilisée, puis est incrémentée

```
Ex : $ declare -i x=9 y
      $ (( y = x++ )) => y reçoit la valeur 9 ; x vaut 10
      $ echo "y=$y x=$x"
      y=9 x=10
      $
```

Post-décrémentation : **var--** la valeur de *var* est d'abord utilisée, puis est décrémentée

```
Ex : $ declare -i x=9 y
```



```
$ (( y = x-- )) => y reçoit la valeur 9 ; x vaut 8
$ echo "y=$y x=$x"
y=9 x=8
$
```

Pré-incrémentation : **++var** la valeur de *var* est d'abord incrémentée, puis est utilisée

```
Ex : $ declare -i x=9 y
$ (( y=++x )) => x vaut 10 ; y reçoit la valeur 10
$ echo "y=$y x=$x"
y=10 x=10
$
```

Pré-décrémentation : **--var** la valeur de *var* est d'abord décrémentée, puis est utilisée

```
Ex : $ declare -i x=9 y
$ (( y=--x )) => x vaut 8 ; y reçoit la valeur 8
$ echo "y=$y x=$x"
y=8 x=8
$
```

Les autres opérateurs s'appliquent à des *expressions arithmétiques*.

Moins unaire : *- expr_arith*

Addition : *expr_arith + expr_arith*

Soustraction : *expr_arith - expr_arith*

Multiplication : *expr_arith * expr_arith*

Division entière : *expr_arith / expr_arith*

Reste de division entière : *expr_arith % expr_arith*

14. OUTILS SUPPLEMENTAIRES / OUTILS D'ASSISTANCE POUR LA CREATION DE SCRIPTS

14.1. EXPRESSIONS RATIONNELLES : OUTIL GREP.

Commande unix **grep** :

Cette commande affiche sur sa sortie standard l'ensemble des lignes contenant une chaîne de caractères spécifiée en argument, lignes appartenant à un ou plusieurs fichiers texte (ou par défaut, son entrée standard).

La syntaxe de cette commande est : **grep** [*option(s)*] *chaîne_cherchée* [*fich_texte1 ...*]

Soit le fichier *pass* contenant les cinq lignes suivantes :

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bertrand:x:101:100::/home/bertrand:/bin/bash
albert:x:102:100::/home/albert:/bin/bash
logware:x:103:100::/home/logware:/bin/bash
```



La commande ci-dessous affiche toutes les lignes du fichier *pass* contenant la chaîne *logware*.

Ex : `$ grep logware pass`

```
logware:x:103:100::/home/logware:/bin/bash
$
```

Attention : **grep** recherche une *chaîne de caractères* et non un *mot*.

Ex : `$ grep bert pass`

```
bertrand:x:101:100::/home/bertrand:/bin/bash
albert:x:102:100::/home/albert:/bin/bash
$
```

La commande affiche toutes les lignes contenant la chaîne *bert* (et non le mot *bert*).

Si l'on souhaite la chaîne cherchée en début de ligne, on utilisera la syntaxe "**^***chaîne_cherchée*". Si on la veut en fin de ligne : "*chaîne_cherchée***\$**"

Ex : `$ grep "^bert" pass`

```
bertrand:x:101:100::/home/bertrand:/bin/bash
$
```

La commande unix **grep** positionne un code de retour

- égal à **0** pour indiquer qu'une ou plusieurs lignes ont été trouvées
- égal à **1** pour indiquer qu'aucune ligne n'a été trouvée
- égal à **2** pour indiquer la présence d'une erreur de syntaxe ou qu'un fichier mentionné en argument est inaccessible.



Ex : \$ grep logware pass

```
logware:x:103:100::/home/logware:/bin/bash
echo $?
$ 0
$
$ grep toto pass
$
$ echo $?
1 => la chaîne toto n'est pas présente dans pass
$
$ grep logware turlututu
grep: turlututu: Aucun fichier ou répertoire de ce type
$
$ echo $?
2 => le fichier turlututu n'existe pas !
$
```

Exercice : Ecrire un programme Shell *connu* prenant en argument un nom d'utilisateur qui affiche 0 s'il est enregistré dans le fichier *pass*, 1 sinon.

```
Ex : $ connu bert
1
$ connu albert
```

Caractères spéciaux

En plus des caractères de redirection des flux standards de données, le Shell possèdent des caractères dont la signification est très... spéciale, les voici regroupés dans le tableau suivant :

Caractère	Description
*	Métacaractère qui remplace n'importe quelle chaîne de caractères (même vide). Exemple : cp * DATA copie tous les fichiers dans le répertoire DATA.
?	Métacaractère qui remplace un caractère quelconque.
;	Permet de séparer plusieurs commandes écrites sur une même ligne. Exemple : cp *.c DATA; tar cvf data.tar DATA copie tous les fichiers d'extention .c dans le répertoire DATA et les archive dans le fichier <i>data.tar</i> .
()	Regroupe des commandes. Exemple :(echo "Liste :"; ls) > liste.txt écrit la chaîne <i>Liste :</i> et la liste des fichiers du répertoire courant dans le fichier <i>liste.txt</i> .
&	Permet le lancement d'un processus en arrière plan. Cela permet d'exécuter d'autres commandes pendant qu'un processus est en marche. Exemple : netscape&.
	Permet la communication par tube entre deux commandes. Exemple : ls -l file la commande de listage des fichiers du répertoire (ls) envoie chacun d'eux à la commande qui permet de connaître le type d'un fichier (file).



#	Introduit un commentaire. Donc tout ce qui suit ce caractère dans une ligne est ignoré par le Shell. Exemple : # ceci est un commentaire.
\	Déspecialise le caractère qui suit. C'est-à-dire que si le caractère qui suit celui là est un caractère spécial alors le Shell l'ignorera. Exemple : echo Bon*jour affiche <i>bon*jour</i> à l'écran.
'...'	Définit une chaîne de caractères qui ne sera pas évaluée par le Shell. Exemple : echo '*?&' affiche sur la sortie standard les caractères spéciaux *?& sans les interpréter.
"..."	Définit une chaîne de caractères dont les variables seront évaluées par le Shell. Exemple : echo "Vous êtes \$USER." affiche <i>Vous êtes</i> + la valeur de la variable \$USER.
`...`	Définit une chaîne de caractères qui sera interprétée comme une commande et remplacée par la chaîne qui serait renvoyée sur la sortie standard à l'exécution de la dite commande. Exemple : echo `pwd` >> liste.txt écrit à la fin du fichier le chemin et le nom du répertoire courant. Le caractère spécial utilisé s'obtient par la combinaison de touche : <i>AltGr + 7</i> (c'est l'accent grave).

14.2. RECHERCHE ET TRAITEMENT DE FICHIERS : OUTIL FIND.

find est un utilitaire UNIX de longue date. Son objectif est de parcourir de façon récursive un ou plusieurs répertoires et d'y trouver des fichiers correspondant à un certain ensemble de critères. Bien qu'il soit très utile, sa syntaxe est vraiment complexe, et l'utiliser requiert une certaine pratique. La syntaxe générale est :

```
find [options] [répertoires] [critère] [action]
```

Si vous ne spécifiez aucun répertoire, **find** recherchera dans le répertoire courant. L'absence de critère équivaut à « vrai », et donc tous les fichiers seront trouvés. Les options, les actions et les critères possibles sont si nombreux que nous n'en mentionnerons que quelques-uns. Commençons par les options :

- -xdev : ne pas étendre la recherche aux répertoires se trouvant sur d'autres systèmes de fichiers.
- -mindepth <n> : descendre d'au moins n niveaux au-dessous du répertoire de recherche avant de chercher des fichiers.
- -maxdepth <n> : rechercher les fichiers se trouvant au plus n niveaux au-dessous du répertoire de recherche.
- -follow : suivre les liens symboliques s'il pointent vers des répertoires. Par défaut, **find** ne les suivra pas.
- -daystart : quand il est fait usage de tests relatifs à la date et à l'heure (voir ci-dessous), prendre le début de la journée courante comme repère au lieu de la marque par défaut (24 heures avant l'heure courante).



15. MANIPULATION DE FLUX DE TEXTE AVEC SED

Sed est un éditeur ligne non interactif. Il reçoit du texte en entrée, que ce soit à partir de stdin ou d'un fichier, réalise certaines opérations sur les lignes spécifiées de l'entrée, une ligne à la fois, puis sort le résultat vers stdout ou vers un fichier. A l'intérieur d'un script Shell, sed est habituellement un des différents outils composant un tube.

Sed détermine sur quelles lignes de son entrée il va opérer à partir de *l'ensemble d'adresses* qui lui est passé. [1] Cette plage d'adresses est définie soit par des numéros de ligne soit par un motif à rechercher. Par exemple, *3d* indique à sed qu'il doit supprimer la ligne 3 de l'entrée, et */windows/d* dit à sed que vous voulez que toutes les lignes de l'entrée contenant << windows >> soient supprimées.

De toutes les opérations de la boîte à outils sed, nous nous occuperons principalement des trois les plus communément utilisées. Il s'agit de **printing** (NdT: affichage vers stdout), **deletion** (NdT: suppression) et **substitution** (NdT: euh... substitution :).

Tableau : Opérateurs sed basiques

Opérateur	Nom	Effet
<i>[plage-d-adresses]/p</i>	print	Affiche [la plage d'adresse spécifiée]
<i>[plage-d-adresses]/d</i>	delete	Supprime [la plage d'adresse spécifiée]
<i>s/motif1/motif2/</i>	substitute	Substitue motif2 à la première instance de motif1 sur une ligne
<i>[plage-d-adresses]/s/motif1/motif2/</i>	substitute	Substitue motif2 par la première instance de motif1 sur une ligne, en restant dans la <i>plage-d-adresses</i>
<i>[plage-d-adresses]/y/motif1/motif2/</i>	transform	remplace tout caractère de motif1 avec le caractère correspondant dans motif2, en restant dans la <i>plage-d-adresses</i> (équivalent de tr)
<i>g</i>	global	Opère sur <i>chaque</i> correspondance du motif à l'intérieur de chaque ligne d'entrée de la plage d'adresse concernée

La substitution opère seulement sur la première instance de la correspondance d'un motif à l'intérieur de chaque ligne, sauf si l'opérateur *g* (*global*) est ajouté à la commande *substitute*.

A partir de la ligne de commande et dans un script Shell, une opération sed peut nécessiter de mettre entre guillemets et d'utiliser certaines options.

```
sed -e '/^$/d' $nomfichier
# L'option -e fait que la chaîne de caractère suivante est interprétée comme
# une instruction d'édition.
# (Si vous passez une seule instruction à "sed", le "-e" est optionnel.)
```



```
# Les guillemets "forts" (") empêchent les caractères de l'ER compris dans
#+ l'instruction d'être interprétés comme des caractères spéciaux par le corps
#+ du script.
# (Ceci réserve l'expansion de l'ER de l'instruction à sed.)
#
# Opère sur le texte contenu dans le fichier $nomfichier.
```

Dans certain cas, une commande d'édition **sed** ne fonctionnera pas avec des guillemets simples.

```
nomfichier=fichier1.txt
modele=BEGIN

sed "/^$motif/d" "$nomfichier" # fonctionne comme indiqué
# sed '/^$motif/d' "$nomfichier" a des résultats inattendus.
# Dans cette instance, avec des guillemets forts (' ... '),
#+ "$modele" ne sera pas étendu en "BEGIN".
```

Sed utilise l'option **-e** pour spécifier que la chaîne suivante est une instruction ou un ensemble d'instructions. Si la chaîne ne contient qu'une seule instruction, alors cette option peut être omise.

```
sed -n '/xzy/p' $nomfichier
# L'option -n indique à sed d'afficher seulement les lignes correspondant au
#+ motif.
# Sinon toutes les lignes en entrée s'afficheront.
# L'option -e est inutile ici car il y a une seule instruction d'édition.
```



16. AUTOMATISATION DE TACHES AVEC AWK

ELEMENTS GENERAUX DE PROGRAMMATION AVEC AWK.

Awk est un langage de manipulation de texte plein de fonctionnalités avec une syntaxe proche du **C**. Alors qu'il possède un ensemble impressionnant d'opérateurs et de fonctionnalités, nous n'en couvrirons que quelques-uns, les plus utiles pour l'écriture de scripts Shell.

Awk casse chaque ligne d'entrée en *champs*. Par défaut, un champ est une chaîne de caractères consécutifs délimités par des [espaces](#), bien qu'il existe des options pour changer le délimiteur. Awk analyse et opère sur chaque champ. Ceci rend awk idéal pour gérer des fichiers texte structurés, particulièrement des tableaux, des données organisées en ensembles cohérents, tels que des lignes et des colonnes.

Des guillemets forts (guillemets simples) et des accolades entourent les segments de code awk dans un script Shell.

```
awk '{print $3}' $nomfichier
# Affiche le champ #3 du fichier $nomfichier sur stdout.

awk '{print $1 $5 $6}' $nomfichier
# Affiche les champs #1, #5 et #6 du fichier $nomfichier.
```

Nous venons juste de voir la commande awk **print** en action. Les seules autres fonctionnalités de awk que nous avons besoin de gérer ici sont des variables. Awk gère les variables de façon similaire aux scripts Shell, quoiqu'avec un peu plus de flexibilité.

```
{ total += ${numero_colonne} }
```

Ceci ajoute la valeur de *numero_colonne* au total << total >>. Finalement, pour afficher << total >>, il existe un bloc de commandes **END**, exécuté après que le script ait opéré sur toute son entrée.

```
END { print total }
```

Correspondant au **END**, il existe **BEGIN**, pour un bloc de code à exécuter avant que awk ne commence son travail sur son entrée.

L'exemple suivant illustre comment **awk** ajout des outils d'analyse de texte dans un script Shell.



Exemple : Compteur sur le nombre d'occurrences des lettres

```
#!/bin/sh
# letter-count.sh: Compter les occurrences des lettres d'un fichier texte.
#
# Script de nyal (nyal@voila.fr).
# Utilisé avec sa permission.
# Nouveaux commentaires par l'auteur du document.

INIT_TAB_AWK=""
# Paramètre pour initialiser le script awk.
compteur=0
FICHIER_A_ANALYSER=$1

E_PARAMERR=65

usage()
{
    echo "Usage: letter-count.sh fichier lettres" 2>&1
    # Par exemple: ./letter-count.sh nom_fichier a b c
    exit $E_PARAMERR # Pas assez d'arguments passé au script.
}

if [ ! -f "$1" ] ; then
    echo "$1: Fichier inconnu." 2>&1
    usage    # Affiche le message d'usage et quitte.
fi

if [ -z "$2" ] ; then
    echo "$2: Aucune lettre spécifiée." 2>&1
    usage
fi

shift          # Lettres spécifiées.
for lettre in `echo $@` # Pour chacune...
do
    INIT_TAB_AWK="$INIT_TAB_AWK tableau_recherche[${compteur}] = \"${lettre}\"; tableau_final[${compteur}] = 0; "
    # A passer au script awk ci-dessous.
    compteur=`expr $compteur + 1`
done

# DEBUG:
# echo $INIT_TAB_AWK;

cat $FICHIER_A_ANALYSER |
# Envoyer le fichier cible au script awk suivant.

# -----
awk -v tableau_recherche=0 -v tableau_final=0 -v tab=0 -v nb_letter=0 -v chara=0 -v caractere2=0 \
"BEGIN { $INIT_TAB_AWK } \\"
```



```

{ split(\$0, tab, "\\"); \
for (caractere in tab) \
{ for (caractere2 in tableau_recherche) \
{ if (tableau_recherche[caractere2] == tab[caractere]) { tableau_final[caractere2]++ } } } \
END { for (caractere in tableau_final) \
{ print tableau_recherche[caractere] \" => \" tableau_final[caractere] } }"
# -----
# Rien de très compliqué, seulement...
#+ boucles for, tests if et quelques fonctions spécialisées.

exit $?

```

****FIN****

CONTACTS

Le Service formation de LOGWARE se tient à votre disposition pour répondre à toutes vos questions : formation@logware.fr.



58A rue du dessous des berges 75013 Paris
Tél. : 00 33 1 53 94 71 20 / Fax : 00 33 1 53 94 71 29
N° Organisme formation : 11754353975
Email : formation@logware.fr
Site : www.logware.fr

